Plan de Pruebas, Guía de Pruebas y Especificación de Test Use Cases

Arquitectura e Ingeniería de Software | Plataforma C# (.NET 10) & WPF | Validación Integral de Requerimientos (RF01–RF16, RNF01–RNF13), Persistencia Políglota (Oracle 21c XE + Neo4j 5 en Docker), Fachada, GraphQL y Automatización con Robot

Version:	1.0 (Alineado al Stack Tecnológico .NET 10 / C# WPF / Docker)
Author:	Eric Charpenel Martínez Ethan Gabriel Covarrubias Gutiérrez Alejandro Hernández Ramos
Institution & Course:	Universidad La Salle México — Facultad de Ingeniería (Grupo 700)
Professor / Reviewer:	Juan Damián Silva Galindo
Technical & Quality Baseline:	IEEE 829 / ISO/IEC/IEEE 29119 (Software Testing) | BABOK v3 | ISO/IEC 20000 | ISO 31000 & COSO-ERM | Propiedades Transaccionales ACID (Oracle 21c XE) & Metodología FIFO
Technology Stack Under Test:	**Plataforma:** C# / .NET 10 | **GUI:** WPF (XAML), `MaterialDesignThemes 5.3.1`, `Microsoft.Web.WebView2 1.0.3967.48` | **Acceso a Datos:** `Dapper 2.1.72`, `Oracle.ManagedDataAccess.Core 23.26.200`, `Neo4j.Driver 6.0.0` | **API & Reportes:** ASP.NET Core, `HotChocolate.AspNetCore 16.7.5` (GraphQL), `QuestPDF` | **Infraestructura:** Docker (Oracle Database 21c XE & Neo4j 5)

RELEASE & DISTRIBUTION

Version	Changes	Areas Affected	Comments
1.0	Consolidación del Plan de Pruebas, Guía de Pruebas y Fichas de Test Use Cases (TUC-01 a TUC-18)	5 Épicas / Módulos (UT, IT, ST, RT), Capa de Fachada/GraphQL y Robot Automatizado en C#	Versión homologada para C# (.NET 10) y WPF basada en UC0, UC1 (CU1.1 a CU1.5), RF01–RF16 y RNF01–RNF13

Name	Organisation	Title
Equipo de Desarrollo (Charpenel, Covarrubias, Hernández)	SGRC — Universidad La Salle México	Ingenieros de Software / QA Lead & Test Automation
Juan Damián Silva Galindo	Universidad La Salle México	Profesor Titular — Revisor

TABLE OF CONTENTS (ÍNDICE GENERAL DE NAVEGACIÓN)

Sección / Bloque	Contenido Técnico Detallado
PARTE I — PLAN Y GUÍA DE PRUEBAS	**Estrategia de Validación, Diseño Arquitectónico (.NET 10 / WPF) y Scripts Paso a Paso (UT, IT, ST, RT)**
1. SCOPE	Alcance funcional y no funcional (5 Épicas del SGRC, RF01 a RF16 y RNF01 a RNF13), exclusiones técnicas y supuestos de validación (S-1 a S-6).
2. SYSTEM DESIGN	Arquitectura en C# (.NET 10) y WPF con patrón MVC/Fachada, servidor GraphQL (Hot Chocolate 16.7.5), persistencia políglota en Docker (Oracle 21c XE + Neo4j 5), modelo de dominio (7 clases) y máquina de estados UI.
3. OBJECTIVES	Niveles de prueba (UT, IT, ST, RT), cobertura de escenarios positivos, negativos, seguridad, rendimiento e integración, y criterios de salida (Exit Criteria).
4. ROLES	Responsabilidades de ejecución (Desarrolladores .NET, QA Lead / Tester Automation con Robot FlaUI/xUnit, Administrador y Voluntario).
5. RECURRENCE	Cadencia de ejecución en el flujo Kanban (5 Épicas), pruebas continuas en versionador, validación en contenedores Docker y ejecución del Robot Automatizado.
6. SCENARIOS (GUÍA DE PRUEBAS)	Datos de prueba maestros (Master Test Data), 12 scripts Unitarios (UT-01 a UT-12), 6 scripts Integrales (IT-01 a IT-06), 5 scripts de Sistema (ST-01 a ST-05) y 2 scripts de Regresión/Robot (RT-01 y RT-02).
7. TRACEABILITY	Matrices de trazabilidad completas: Caso de Uso Nivel 0 (UC0) y Nivel 1 (UC1) vs. Scripts, Requerimientos Funcionales (RF01–RF16) y No Funcionales (RNF01–RNF13) vs. Scripts.
PARTE II — TEST USE CASES (TUC)	**Catálogo de 18 Casos de Prueba en Ficha Estructurada (TUC-01 a TUC-18)**
Bloque 1 (TUC-01 a TUC-03)	Autenticación de usuarios (UC1.2 / UC0 Paso 1), bloqueo al 3.er intento fallido (1A) y CRUD de cuentas de voluntarios con modal de confirmación (RF13, RNF04).
Bloque 2 (TUC-04 y TUC-05)	Gestión de donantes, vinculación al historial (UC1.1 / UC0 Paso 2) y validación de excepciones por datos inválidos, duplicados y campos vacíos (1a, 2a, 3a).
Bloque 3 (TUC-06 a TUC-11)	Registro de insumos en ≤ 3 clics (UC1.3), Rollback por fecha vencida (2A) y bloqueo en WPF por letras (3A), edición de lotes (RF03), bajas por merma (RF15), búsqueda ≤ 2 s (RNF07) y semáforo con vista `Simulación` (RNF01).
Bloque 4 (TUC-12 a TUC-15)	Padrón de beneficiarios (RF08), entregas logísticas guiadas por FIFO con notificación Observer y mapeo en Neo4j (UC1.5), bloqueo por violación FIFO (4A) y restauración de devoluciones (RF16).
Bloque 5 (TUC-16 a TUC-18)	Reportes de stock por categoría y exportación PDF (QuestPDF) y CSV en ≤ 5 s (UC1.4), excepciones de filtros (1a, 2a, 3a) y persistencia ACID ante cierre inesperado (5A), respaldo 24h y límite de RAM ≤ 512 MB.

PARTE I — PLAN DE PRUEBAS Y GUÍA DE PRUEBAS (SGRC)

1. SCOPE

Se presenta el Plan de Pruebas, los Casos de Prueba (Test Use Cases) y la Guía de Pruebas del **Sistema de Gestión de Recursos Comunitarios (SGRC)**, desarrollado en **C# sobre la plataforma .NET 10 con interfaz gráfica de escritorio en WPF (Windows Presentation Foundation)**, diseñado para optimizar la operación, trazabilidad y distribución de insumos perecederos en bancos de alimentos conforme al Caso de Uso Principal de Nivel 0 (`UC0: Gestionar Banco de Alimentos`), los 5 Casos de Uso de Nivel 1 (`UC1.1` a `UC1.5`), sus flujos alternos de excepción, los 16 Requerimientos Funcionales (RF01 a Nuevo RF16), los 13 Requerimientos No Funcionales (RNF01 a RNF13) y el análisis de diagnóstico PIECES.

Dentro del alcance:

- **Módulo 1 — Seguridad, Autenticación y Control de Acceso (RF01, RF13):** Validación de credenciales para los roles de `Administrador` y `Voluntario` en `LoginView.xaml` mediante `Dapper` y `Oracle.ManagedDataAccess.Core`, detección automática de rol, bloqueo temporal de cuenta tras 3 intentos fallidos consecutivos (Flujo Alterno 1A) y gestión completa (CRUD: crear, editar y eliminar) de cuentas de acceso del personal voluntario con confirmación modal previa (RNF04).
- **Módulo 2 — Gestión de Inventario Perecedero y Validaciones de Entrada en WPF (RF02, RF03, RF04, RF05, Nuevo RF14, Nuevo RF15):** Captura de ingresos físicos de insumos perecederos (nombre, categoría, lote, cantidad y fecha de caducidad) a través de la capa de Fachada, edición por errores de captura en `Gestión de Lote`, baja de productos caducados o mermas clasificando su motivo específico para trazabilidad de pérdidas, búsqueda filtrada por nombre, categoría o caducidad, y bloqueo estricto de fechas vencidas (Rollback en Oracle Database 21c XE — Flujo Alterno 2A) o caracteres alfabéticos en campos numéricos (desactivación de botón y borde rojo en controles XAML de WPF con `MaterialDesignThemes` — Flujo Alterno 3A).
- **Módulo 3 — Motor FIFO, Semáforo de Urgencia y Restauración de Entregas (RF06 Corregido, RF10, Nuevo RF16, RNF01):** Cálculo y renderizado del código de colores semafórico (rojo crítico, amarillo preventivo, verde óptimo) en los controles `DataGrid` de `DashboardView.xaml` e `InventarioView.xaml`, consulta de lotes próximos a caducar, salida guiada obligatoriamente por el método *First-In, First-Out* (FIFO) autoseleccionando el lote más antiguo (Flujo Alterno 4A), notificación mediante el patrón *Observer* y restauración de insumos derivados de entregas canceladas o devoluciones respetando su posición cronológica original.
- **Módulo 4 — Red Logística de Donantes, Beneficiarios, GraphQL y Grafos en Neo4j (RF07, RF08, RF09, RF10, RNF11, RNF13):** Registro de nuevos donantes (organización o persona, teléfono, correo) evitando duplicados (`UC1.1`) y vinculación de cada lote entrante para conservar su historial de aportaciones; registro de beneficiarios (nombre del representante familiar y número de dependientes); mapeo, consulta y visualización (apoyada en `Neo4j.Driver 6.0.0`, `WebView2` y consultas vía `HotChocolate GraphQL`) de la red de distribución logística entre nodos de Donantes y Beneficiarios en el motor de grafos **Neo4j 5** desde `LogisticaView.xaml`.

- **Módulo 5 — Consultas Agregadas, Reportes Ejecutivos (QuestPDF / CSV) y Atributos de Calidad (RF11, RF12, RNF02 a RNF12):** Consulta de stock total agrupado por categorías de alimentos, exportación dinámica de reportes históricos de inventario y logística en formatos **PDF** (generados con la librería `QuestPDF`) y **CSV** (`System.IO`) en ≤ 5 segundos (hasta 10,000 registros), búsquedas en ≤ 2 segundos, consumo de memoria RAM del proceso .NET 10 ≤ 512 MB, navegación al registro de inventario en ≤ 3 clics, tipografía mínima de 14pt, respaldos locales automáticos cada 24 horas y persistencia transaccional ACID en **Oracle Database 21c XE** ante cierres inesperados (Flujo Alterno 5A).

Fuera del alcance:

- Pruebas del código fuente interno de los motores de bases de datos contenerizados en Docker (núcleo interno de Oracle Database 21c XE y motor interno de Neo4j 5); el alcance de validación cubre la capa de persistencia en C# con `Dapper`, `Oracle.ManagedDataAccess.Core` y `Neo4j.Driver`, la Fachada, los resolvers GraphQL en `Hot Chocolate`, las transacciones ACID, rollbacks y sincronización políglota.
- Pruebas de infraestructura eléctrica externa de la bodega o calibración de básculas físicas; la validación inicia en la captura manual o inyección automatizada por el Robot sobre la interfaz cliente WPF en Windows.

Supuestos y reglas de negocio/validación confirmadas:

- **S-1 (Bloqueo por Autenticación Fallida — Flujo Alterno 1A):** El ingreso de credenciales inválidas deniega el acceso mostrando la ventana `Error de Autenticacion`; al acumular exactamente 3 intentos fallidos consecutivos sobre una misma cuenta, el sistema bloquea temporalmente el acceso de dicho usuario y registra el evento de seguridad.
- **S-2 (Rechazo de Fechas Vencidas y Caracteres Inválidos — Nuevo RF14, Flujos 2A y 3A):** Cualquier intento de registrar un lote con `fecha_caducidad < fecha_actual` dispara una alerta de "Rechazo automático" y ejecuta un `ROLLBACK` transaccional en Oracle Database. Asimismo, ingresar letras o símbolos en campos numéricos (`cantidad`, `numDependientes`) resalta la casilla en rojo en WPF y desactiva preventivamente el botón de guardado.
- **S-3 (Cumplimiento Estricto de la Regla FIFO — RF06, RF10, Flujo 4A):** Al registrar una entrega en `Entrega_Logistica` para un insumo que posee múltiples lotes activos en almacén, el sistema obliga a descontar primero del lote con la `fecha_caducidad` más próxima. Si el voluntario intenta seleccionar un lote más reciente existiendo uno antiguo disponible, el sistema cancela la selección manual, emite una alerta de violación FIFO y autoselecciona el lote correcto.
- **S-4 (Código Semafórico de Caducidad y Simulación Temporal — RNF01):** Las tablas de inventario en WPF clasifican visualmente cada lote según los días restantes para su caducidad respecto a la fecha del sistema (o la fecha proyectada en la ventana `Simulación`): Rojo (Crítico / próximo a vencer), Amarillo (Preventivo / prioridad media) y Verde (Holgado / óptima vigencia).
- **S-5 (Umbrales de Rendimiento y Recursos Locales — RNF07, RNF08, RNF09):** Se establece como línea base de aprobación un tiempo de respuesta ≤ 2.0 segundos en búsquedas filtradas con Dapper (MET-01), ≤ 5.0 segundos en la generación y escritura en disco de reportes PDF (QuestPDF) y CSV con un volumen de hasta 10,000 registros (MET-02), y una huella de memoria RAM total del proceso .NET 10 (Working Set) ≤ 512 MB (MET-03).

- **S-6 (Integridad Políglota y Recuperación ACID — RNF05, RNF06, RNF11, RNF12, RNF13):** Toda operación sobre `Insumo_Perecedero`, `Lote` y `Merma` se ejecuta bajo aislamiento ACID en Oracle Database 21c XE, garantizando que un apagón o cierre abrupto (Flujo 5A) cancele transacciones incompletas sin corromper datos; paralelamente, toda entrega confirmada a través de la Fachada actualiza el nodo y relación correspondiente entre `Donante` y `Beneficiario` en Neo4j 5.

2. SYSTEM DESIGN

El diseño del sistema sometido a prueba corresponde a la arquitectura documentada en Enterprise Architect, el Modelo de Dominio (Diagrama de Clases), el Modelo Lógico/Relacional en Oracle SQL Developer, el Diagrama de Máquina de Estados, los Wireframes de Interfaz de Usuario y la solución en Visual Studio (.NET 10):

- **Arquitectura tecnológica, librerías NuGet y patrones de diseño (RNF10, RNF11, RNF12, RNF13):**
  - **Plataforma y Frontend de Escritorio:** Desarrollado en **C# sobre .NET 10** utilizando **WPF (Windows Presentation Foundation)** bajo arquitectura en capas (**MVC + Patrón Fachada + Patrón Observer**). La interfaz emplea `MaterialDesignThemes 5.3.1` para componentes visuales modernos y diálogos modales, y `Microsoft.Web.WebView2 1.0.3967.48` para visualización web integrada.
  - **Persistencia Políglota Contenerizada en Docker:** Acceso relacional a **Oracle Database 21c XE** mediante el micro-ORM `Dapper 2.1.72` y el proveedor oficial `Oracle.ManagedDataAccess.Core 23.26.200` garantizando transacciones ACID. Acceso al motor de grafos **Neo4j 5** mediante `Neo4j.Driver 6.0.0` ejecutando consultas Cypher para trazabilidad logística.
  - **Capa de Servicios GraphQL y Generación de Reportes:** Servidor web en **ASP.NET Core** integrando `HotChocolate.AspNetCore 16.7.5`, `HotChocolate.AspNetCore.CommandLine 16.7.5` y `HotChocolate.Types.Analyzers 16.7.5` para consultas y resolvers GraphQL (`SGRC.GraphQL.Api`). Exportación de documentos mediante `QuestPDF` (reportes PDF) y `System.IO` / `System.Text` (reportes CSV).
- **Entidades del Modelo de Dominio y Base de Datos:**
  - `Usuario` (Superclase con `idUsuario`, `nombreCompleto`, `correo`, `contraseña`) de la cual heredan `Administrador` (`nivelPrivilegio`, `puestoCargo`) y `Voluntario` (`horasAcumuladas`, `turnoAsignado`).
  - `Insumo_Perecedero` (`idInsumo`, `nmbreInsumo`, `categoria`) asociado 1 a * con `Lote` (`idLote`, `cantidad`, `fecha_caducidad`, `estadoStock` / `Estado_Semaforo`).
  - `Donante` (`idDonante`, `nombreOrganización`, `telefono`, `correo`) asociado 1 a * con `Lote`.
  - `Beneficiario` (`idBeneficiario`, `nombreRepresentante`, `numDependientes`) asociado 1 a * con `Entrega_Logistica` (`idEntrega`, `fechaSalida`, `cantidadEntregada`), la cual vincula al `Usuario` ejecutor y al `Lote` distribuido.
  - `Merma` (`Cantidad`, `Motivo`, `Fecha`) vinculada al `Lote` dado de baja (Nuevo RF15).

- **Flujo de Navegación e Interfaces Gráficas en WPF (Máquina de Estados y Wireframes UI):**
  - `LoginView`: Ventana XAML inicial con controles `txtUser`, `txtPwd` y botón `ENTRAR AL SISTEMA`. Conecta con los modales `Autenticacion Exitosa` (detectando rol Administrador o Voluntario) y `Error de Autenticacion` (botón `OK`).
  - `DashboardView` (`MainWindow - Voluntario / Administrador`): Panel de control central con barra lateral de navegación (botones `Panel de Control`, `Inventario (Oracle)`, `Logistica (Neo4j)`), indicadores resumidos (`Productos por caducar`, `Donaciones Hoy`, `Sistemas de Datos`), botones superiores (`Notificacion`, `Cerrar Sesion`, `Simular Fecha`, `Registrar Ingreso`) y acceso a modales `Registro Exitoso` y `NOTIFICACION` (disparado por el método Observer).
  - `InventarioView`: Vista tabular de lotes en Oracle con código de colores semafórico, filtros de búsqueda, transición hacia `Gestión de Lote` (acciones de Editar / Eliminar con confirmación modal) y botón `Exportar CSV` / `PDF` que dispara el modal `Exito CSV` (`Reporte exportado dinamicamente! Guardado en: PATH`).
  - `LogisticaView`: Interfaz conectada a Neo4j 5 para registro de entregas guiadas por FIFO, alta de donantes/beneficiarios, reingreso de devoluciones y visualización del grafo de distribución.
  - `Simulación` (`MainWindow - VOLUNTARIO - SIMULACION FECHA`): Vista de proyección temporal para evaluar el cambio dinámico del semáforo de caducidad y alertas de urgencia.
- **Actores del sistema (2 actores operativos modelados):**
  - **Voluntario (Actor Principal):** Ejecuta la logística diaria del banco de alimentos (registro de ingresos perecederos, corrección de capturas, búsqueda y consulta de urgencia semafórica, registro de beneficiarios, salidas guiadas por FIFO y restauración de devoluciones).
  - **Administrador (Actor Secundario / Control):** Ejecuta la gestión administrativa y de control del refugio (alta y vinculación de donantes, baja de productos caducados y clasificación de mermas, consulta de stock por categoría, exportación de reportes PDF/CSV y administración CRUD de cuentas de voluntarios).

3. OBJECTIVES

El objetivo del presente documento es establecer el Plan de Pruebas, los Test Use Cases y la Guía de Pruebas detallada cubriendo de forma explícita los cinco tipos de escenarios exigidos para la validación integral del SGRC:

- **Escenarios Positivos (Flujos nominales / Basic Path):** Verificación de que cada paso del Caso de Uso Principal (pasos 1 al 7 del Basic Path de `UC0`), los 5 Casos de Uso de Nivel 1 (`UC1.1` a `UC1.5`) y los 16 Requerimientos Funcionales procesan a través de la Fachada, almacenan en Oracle 21c XE / Neo4j 5 y actualizan en WPF las operaciones válidas del banco de alimentos.
- **Escenarios Negativos (Flujos Alternos 1A a 5A de UC0, Alternos 1a/2a/3a de UC1 y valores límite):** Verificación de rechazos por credenciales incorrectas, donantes duplicados o con datos vacíos, ingreso de fechas de caducidad pasadas (Rollback en Oracle), escritura de caracteres alfabéticos en campos numéricos (bloqueo de botón en WPF), intento de violación de la regla FIFO al despachar lotes nuevos antes que los antiguos, filtros inválidos en reportes e intento de eliminar registros sin confirmar la ventana emergente.

- **Escenarios de Seguridad y Control de Acceso (RBAC y Transaccionalidad):** Validación de bloqueo de cuenta al tercer intento fallido (1A), segregación de permisos entre los roles de `Voluntario` y `Administrador`, parametrización de consultas en Dapper y Neo4j.Driver contra inyección SQL/Cypher, y preservación de las propiedades ACID más respaldos automáticos cada 24 horas (RNF05, RNF06, RNF12).
- **Escenarios de Rendimiento, Usabilidad y Recursos (SLA Técnico):** Validación del tiempo de respuesta en búsquedas filtradas (≤ 2.0 s, RNF07), generación de reportes PDF (QuestPDF) y CSV con hasta 10,000 registros (≤ 5.0 s, RNF08), consumo de memoria RAM en ejecución continua sobre .NET 10 (≤ 512 MB, RNF09), navegabilidad en máximo 3 clics (RNF02) y tipografía ≥ 14pt (RNF03).
- **Escenarios de Integración Políglota, GraphQL y Automatización con Robot (End-to-End):** Validación del flujo completo entre las vistas WPF, la Fachada, los resolvers GraphQL (Hot Chocolate), la persistencia relacional en Oracle Database 21c XE y el mapeo de nodos/relaciones en Neo4j 5, incluyendo la ejecución desatendida mediante el **Robot Automatizado de Pruebas en C# (FlaUI / xUnit)**.

Resumen cuantitativo de la Guía de Pruebas y Criterios de Salida (Exit Criteria):

- **Pruebas Unitarias (UT-01 a UT-12):** 12 scripts (52 pasos) enfocados en reglas de negocio aisladas, validaciones de entrada en WPF (RF14), semáforo (RNF01), motor FIFO (RF10), mermas (RF15), devoluciones (RF16) y CRUD de usuarios/donantes/beneficiarios.
- **Pruebas Integrales (IT-01 a IT-06):** 6 scripts (25 pasos) enfocados en la interacción entre vistas WPF, Fachada/Controladores, transacciones ACID en Oracle con Dapper, grafos en Neo4j, endpoint GraphQL, exportación a disco (QuestPDF/CSV) y flujo E2E de Nivel 0.
- **Pruebas de Sistema (ST-01 a ST-05):** 5 scripts (19 pasos) enfocados en rendimiento de búsquedas y reportes masivos (10,000 registros), perfilado de memoria RAM en .NET 10 (≤ 512 MB), recuperación ante apagón/cierre abrupto (Flujo 5A) y auditoría de usabilidad (≤ 3 clics, ≥ 14pt).
- **Pruebas de Regresión y Robot Automatizado (RT-01 y RT-02):** 2 scripts (14 pasos) que gobiernan la ejecución del Robot Automatizado sobre WPF (destacando el registro exitoso de lote perecedero de extremo a extremo) y la regresión incremental de las 5 Épicas del tablero Kanban.
- **Especificación de Test Use Cases (TUC-01 a TUC-18):** 18 fichas técnicas estructuradas que cubren todos los caminos básicos y alternos de `UC0` y `UC1`.
- **Criterio de aprobación (Exit Criteria):** 100% de los 25 scripts de la Guía y las 18 fichas TUC ejecutados y aprobados sin defectos críticos o altos abiertos, cumplimiento del 100% de las métricas (MET-01 a MET-13) y ejecución exitosa del Robot Automatizado.

4. ROLES

Las pruebas descritas en este plan son ejecutadas y dictaminadas por los siguientes perfiles del proyecto SGRC:

1. **Desarrolladores de Arquitectura, Backend .NET y UI WPF (Eric Charpenel, Ethan Gabriel Covarrubias, Alejandro Hernández):** Ejecución de pruebas unitarias (UT-01 a UT-12) con xUnit/NUnit, depuración de vistas XAML, Fachada y resolvers GraphQL (Hot Chocolate), ajuste de consultas SQL en Dapper (Oracle) y Cypher (Neo4j.Driver), y corrección de defectos en el versionador.
2. **QA Lead & Especialista en Pruebas Automatizadas (Alejandro Hernández Ramos):** Diseño del Plan de Pruebas y Test Use Cases, codificación del **Robot Automatizado de Pruebas en C# (FlaUI)**, ejecución de pruebas integrales (IT), pruebas de estrés/rendimiento (ST), control de métricas (MET-01 a MET-13), evaluación de la Matriz de Riesgos (R1 a R14) y consolidación del entregable de Pruebas.
3. **Usuarios Representantes del Negocio (Validación Operativa UAT):** Actores en los roles de **Voluntario** (operación de piso de bodega, entradas, búsquedas y despachos FIFO) y **Administrador** (control de donantes, mermas, cuentas de usuario y reportes ejecutivos).
4. **Revisor Académico / Auditor de Calidad (Prof. Juan Damián Silva Galindo):** Evaluación formal del cumplimiento de la rúbrica, trazabilidad contra requerimientos en Enterprise Architect y verificación de la ejecución del Robot Automatizado.

5. RECURRENCE

La ejecución de las pruebas se alinea al flujo de trabajo del tablero **Kanban** del proyecto (estructurado en 5 Épicas dentro de las columnas *Backlog* → *Análisis y Diseño* → *Implementación* → *Pruebas QA* → *Hecho*) y al control de cambios en el versionador:

- **Pruebas Unitarias (UT-01 a UT-12):** Ejecución continua en cada confirmación de código (*commit / merge* en el versionador) al desarrollar o modificar clases de las capas `Model`, `View (WPF XAML)`, `Controller / Fachada` y `GraphQL`.
- **Pruebas Integrales (IT-01 a IT-06) y Regresión por Épica (RT-02):** Ejecución obligatoria cada vez que una tarjeta/Épica del tablero Kanban transiciona de la columna *Implementación (Backend/UI)* hacia la columna *Pruebas (QA)* antes de ser marcada como *Hecho (Done)*.
- **Pruebas de Sistema (ST-01 a ST-05) y Ejecución del Robot Automatizado (RT-01):** Ejecución integral en el equipo de escritorio con los contenedores Docker de Oracle Database 21c XE y Neo4j 5 activos previo a la liberación final y demostración del proyecto.
- **Criterios de entrada y suspensión (Entry & Suspension Criteria):** Se requiere contar con la solución .NET 10 compilada, contenedores Docker de Oracle Database 21c XE y Neo4j 5 en ejecución (Precondiciones 1, 2 y 3 del Caso de Uso Principal), y el script de *Master Test Data* cargado. Las pruebas se suspenden inmediatamente si se detecta caída de conexión en cualquiera de los contenedores Docker o si ocurre corrupción en las tablas transaccionales.

6. SCENARIOS (GUÍA DE PRUEBAS PASO A PASO)

Cada script especifica su identificador único, título descriptivo, trazabilidad directa con los Casos de Uso (`UC0` y `UC1.1` a `UC1.5`), Fichas TUC, Requerimientos (RF/RNF), Métricas (MET), Riesgos (R) e Interfaces de Usuario en WPF (UI), además del tipo de escenario, prioridad, rol ejecutor y la tabla detallada de pasos (**Paso, Actividad, Resultado Esperado y Fecha y Hora**). Nomenclatura: `UT` = Unitaria, `IT` = Integral, `ST` = Sistema, `RT` = Regresión / Robot.

Datos de Prueba Maestros (Master Test Data)

ID Dato / Entidad	Descripción y Parámetros de Configuración en Ambiente de Pruebas (Oracle 21c XE & Neo4j 5)
USR-ADM-01 (Administrador)	Cuenta activa en tabla `Usuario` / `Administrador`: `idUsuario = 1`, `nombreCompleto = "Carlos Mendoza"`, `correo = "admin@sgrc.org"`, `txtUser = "admin"`, `txtPwd = "Admin2026!"`, `nivelPrivilegio = "TOTAL"`, `puestoCargo = "Coordinador General"`.
USR-VOL-01 / 02 (Voluntarios)	Cuenta activa en `Usuario` / `Voluntario`: `idUsuario = 2`, `nombreCompleto = "Laura Torres"`, `correo = "ltorres@sgrc.org"`, `txtUser = "voluntario1"`, `txtPwd = "Vol2026!"`, `horasAcumuladas = 48`, `turnoAsignado = "Matutino"`. Cuenta de prueba para baja/edición: `USR-VOL-02` (`txtUser = "vol_temp"`).
DON-101 / DON-102 (Donantes)	`DON-101`: `idDonante = 101`, `nombreOrganización = "Fundación Alimentar México A.C."`, `telefono = "5551234567"`, `correo = "contacto@alimentarmx.org"`. `DON-102`: `idDonante = 102`, `nombreOrganización = "Central de Abastos CDMX - Nave I"`, `telefono = "5559876543"`, `correo = "donaciones@centralcdmx.mx"`.
BEN-201 / BEN-202 (Beneficiarios)	`BEN-201`: `idBeneficiario = 201`, `nombreRepresentante = "María Guadalupe Hernández"`, `numDependientes = 5`. `BEN-202`: `idBeneficiario = 202`, `nombreRepresentante = "Roberto Sánchez Pérez"`, `numDependientes = 3`.
INS-01 / INS-02 (Insumos Perecederos)	`INS-01`: `idInsumo = 1`, `nmbreInsumo = "Leche Entera Pasteurizada 1L"`, `categoria = "Lácteos"`. `INS-02`: `idInsumo = 2`, `nmbreInsumo = "Manzana Roja Fresca Kg"`, `categoria = "Frutas y Verduras"`.
LOT-501 / 502 / 503 (Lotes Activos FIFO)	Lotes asociados a `INS-01 (Leche 1L)` y `DON-101`: • `LOT-501` (Antiguo / Crítico): `cantidad = 50`, `fecha_caducidad = Fecha_Actual + 2 días`, `estadoStock = "ROJO_CRITICO"`. • `LOT-502` (Intermedio / Preventivo): `cantidad = 80`, `fecha_caducidad = Fecha_Actual + 6 días`, `estadoStock = "AMARILLO_PREVENTIVO"`. • `LOT-503` (Reciente / Holgado): `cantidad = 120`, `fecha_caducidad = Fecha_Actual + 25 días`, `estadoStock = "VERDE_OPTIMO"`.
LOT-504 (Caducado) & DATOS-INVALIDOS	`LOT-504`: Lote que alcanza vencimiento en prueba de simulación para dar de baja como merma (`Motivo = "Caducidad natural en anaquel"`). Entradas inválidas para RF14: `fecha_vencida = "01/01/2024"` y `cantidad_alfabetica = "Cincuenta50ABC"`.
DATASET-10K (Volumen Rendimiento)	Script semilla en Oracle Database 21c XE y Neo4j 5 con `10,000 registros` históricos de lotes y entregas logísticas distribuidos en 6 categorías de alimentos para pruebas ST-01 y ST-02.

PRUEBAS UNITARIAS (UT)

UT-01. Autenticación de usuarios por rol (Voluntario / Administrador) y bloqueo al 3.er intento fallido

> **Trazabilidad:** UC1.2 (Basic Path, 1a, 2a) | UC0 (Paso 1, Flujo Alterno 1A) | TUC-01, TUC-02 | RF01 | MET-12 | Riesgo: R10 | UI: `LoginView.xaml`, `Autenticacion Exitosa`, `Error de Autenticacion` | **Escenario:** Positivo / Negativo / Seguridad | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Abrir el ejecutable WPF del SGRC en `LoginView`, ingresar en `txtUser` el valor `"voluntario1"`, en `txtPwd` el valor `"Vol2026!"` y presionar el botón `ENTRAR AL SISTEMA`.	El sistema valida las credenciales vía Dapper contra la tabla `Usuario` en Oracle, despliega el diálogo `Autenticacion Exitosa` con el texto *"Bienvenido al sistema. Rol detectado: Voluntario"* y al pulsar `OK` redirige a `MainWindow - Voluntario`.	
2	Cerrar sesión desde `DashboardView` e iniciar sesión en `LoginView` con las credenciales de `USR-ADM-01` (`txtUser = "admin"`, `txtPwd = "Admin2026!"`).	El sistema despliega el modal `Autenticacion Exitosa` indicando *"Rol detectado: Administrador"* y habilita en el panel WPF los controles exclusivos de gestión de cuentas, donantes y bajas por merma.	
3	En `LoginView`, ingresar el usuario válido `"voluntario1"` con una contraseña incorrecta (`"ClaveErronea1"`) y presionar `ENTRAR AL SISTEMA` (Intento fallido 1 de 3 — UC1.2 Alterno 2a).	El sistema deniega el acceso y despliega la ventana emergente `Error de Autenticacion` con el mensaje *"Usuario o contraseña incorrectos. Por favor, intente de nuevo."* Al hacer clic en `OK` retorna a `LoginView`.	
4	Repetir el ingreso de contraseña incorrecta por segunda y tercera vez consecutiva sobre la cuenta `"voluntario1"` (Intentos fallidos 2 y 3 — UC0 Flujo Alterno 1A).	Al tercer error consecutivo, el sistema bloquea temporalmente la cuenta `voluntario1`, notifica el bloqueo por seguridad e impide el acceso incluso si en un 4.º intento inmediato se introduce la contraseña correcta.	
5	Intentar presionar `ENTRAR AL SISTEMA` dejando los campos `txtUser` o `txtPwd` completamente vacíos (UC1.2 Alterno 1a).	El sistema valida campos requeridos en la vista WPF sin enviar consulta nula a Oracle y solicita completar usuario y contraseña.	

UT-02. Gestión de cuentas de acceso para personal voluntario (CRUD y confirmación modal)

> **Trazabilidad:** UC0 | TUC-03 | RF13, RNF04 | MET-09, MET-12 | Riesgo: R10 | UI: `DashboardView.xaml (Admin)` | **Escenario:** Positivo / Negativo / Seguridad | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Administrador`, abrir el módulo de cuentas de personal y crear un nuevo voluntario: `nombreCompleto = "Pedro Ramírez"`, `correo = "pramirez@sgrc.org"`, `contraseña = "Pedro2026!"`, `turnoAsignado = "Vespertino"`.	El sistema inserta el registro mediante Dapper en las tablas `Usuario` y `Voluntario` en Oracle con `horasAcumuladas = 0` y muestra mensaje de confirmación de alta.	
2	Seleccionar la cuenta creada, editar el campo `turnoAsignado` a `"Mixto"` y actualizar `horasAcumuladas = 12`.	Los cambios se persisten en Oracle y se reflejan de inmediato en el `DataGrid` de usuarios activos en WPF.	
3	Seleccionar la cuenta de prueba `USR-VOL-02 ("vol_temp")`, presionar el botón `Eliminar` y en la ventana emergente de confirmación (RNF04) seleccionar `Cancelar`.	El sistema despliega obligatoriamente el diálogo modal de confirmación antes de procesar la baja; al cancelar, la cuenta permanece intacta en la base de datos.	
4	Presionar nuevamente `Eliminar` sobre `USR-VOL-02` y hacer clic en `Confirmar / OK` en la ventana emergente.	La cuenta se elimina/inactiva en Oracle y ya no permite iniciar sesión en `LoginView`.	
5	Iniciar sesión como `Voluntario` e intentar acceder a las funciones de creación o eliminación de cuentas de usuario.	Acceso restringido: los controles de administración de cuentas no están habilitados para el rol de Voluntario.	

UT-03. Registro de ingreso de inventario perecedero y validación estricta de datos de entrada en WPF

> **Trazabilidad:** UC1.3 (Basic Path, 1a, 2a) | UC0 (Paso 3, Flujos Alternos 2A y 3A) | TUC-06, TUC-07 | RF02, Nuevo RF14, RNF12 | MET-10, MET-12 | Riesgo: R3, R4 | UI: `DashboardView.xaml`, `InventarioView.xaml`, `Registro Exitoso` | **Escenario:** Positivo / Negativo / Valores Límite | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Voluntario`, hacer clic en `Registrar Ingreso` y capturar un lote válido: Nombre `"Arroz Blanco 1Kg"`, Categoría `"Granos y Cereales"`, Lote `"LOT-601"`, Cantidad `100`, Fecha de caducidad futura (`+60 días`) y Donante `DON-101`.	El sistema valida la entrada en la Fachada, guarda el registro en las tablas `Insumo_Perecedero` y `Lote` en Oracle con estado inicial verde, crea la relación del donante en Neo4j y despliega el modal `Registro Exitoso`; al pulsar `OK` retorna a `DashboardView`.	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	Intentar registrar un nuevo lote ingresando una fecha de caducidad pasada (ej. `01/01/2024` o el día de ayer — UC0 Flujo Alterno 2A, UC1.3 Alterno 1a y Nuevo RF14).	El sistema aborta el registro, ejecuta `Rollback` en Oracle Database 21c XE impidiendo la inserción y lanza la alerta en pantalla de *"Rechazo automático: Fecha de caducidad vencida o inválida"*.	
3	En el control `TextBox` numérico de `Cantidad`, escribir caracteres alfabéticos o símbolos (ej. `"Cincuenta50ABC"` o `"-25"` — UC0 Flujo Alterno 3A y Nuevo RF14).	La validación en tiempo real de WPF (`MaterialDesignThemes` / `ValidationRule`) resalta la casilla de cantidad en color **rojo** y **desactiva inmediatamente el botón de guardado (`IsEnabled = false`)**, impidiendo el envío del formulario.	
4	Corregir el campo `Cantidad` colocando un entero positivo válido (`50`) y una fecha de caducidad igual al límite inferior permitido (`Fecha_Actual + 1 día`).	El borde rojo desaparece, el botón de guardado se reactiva y el lote se registra exitosamente clasificándose de inmediato en estado crítico por su cercanía de vencimiento.	

UT-04. Modificación de datos de productos e insumos por error de captura

> **Trazabilidad:** UC1.3 | UC0 | TUC-08 | RF03, Nuevo RF14 | MET-10, MET-12 | Riesgo: R3 | UI: `InventarioView.xaml`, `Gestión de Lote` | **Escenario:** Positivo / Negativo | **Prioridad:** Media | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	En `InventarioView`, seleccionar el lote recién creado `LOT-601` y hacer clic en el botón `Editar / Eliminar` para transicionar al estado `Gestión de Lote`.	El sistema abre la vista `Gestión de Lote` en WPF precargando los datos actuales del producto (Nombre, Categoría, Cantidad = 100, Fecha de caducidad).	
2	Modificar la cantidad de `100` a `95` (simulando corrección por error de conteo físico en recepción) y hacer clic en `Confirmar Acción`.	El sistema actualiza el registro en Oracle Database mediante Dapper bajo transacción ACID, retorna a `InventarioView` y muestra la cantidad actualizada en `95` unidades.	
3	En `Gestión de Lote`, intentar modificar la fecha de caducidad por una fecha vencida o escribir letras en el campo de cantidad.	Las reglas del Nuevo RF14 se aplican igualmente en la edición: se bloquea el guardado en WPF y se conserva el valor válido previo sin alterar la base de datos.	

UT-05. Baja de productos caducados y clasificación obligatoria del motivo de merma

> **Trazabilidad:** UC0 | TUC-09 | RF04, Nuevo RF15, RNF04 | MET-09, MET-11, MET-12 | Riesgo: R7, R8 | UI: `InventarioView.xaml`, `Gestión de Lote` | **Escenario:** Positivo / Negativo | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Administrador`, localizar en `InventarioView` el lote vencido/dañado `LOT-504`, entrar a `Gestión de Lote` e intentar procesarlo como baja sin seleccionar ni escribir un motivo de merma.	El sistema bloquea la baja e indica que es obligatorio clasificar y registrar el motivo específico de la merma (Nuevo RF15).	
2	Seleccionar la clasificación de merma `"Caducidad natural en anaquel"` (o `"Empaque dañado en bodega"`), indicar la cantidad afectada y presionar `Confirmar Acción`.	El sistema despliega la ventana emergente de confirmación de eliminación/baja (RNF04) advirtiendo sobre el descuento definitivo del inventario activo.	
3	Confirmar la operación en la ventana emergente.	El lote `LOT-504` se da de baja del stock disponible en la tabla `Lote` y se inserta un registro histórico en la tabla `Merma` con su `Cantidad`, `Motivo`, `Fecha` y vínculo al `Lote` y `Donante` origen.	
4	Intentar dar de baja definitiva un lote utilizando una cuenta con rol de `Voluntario`.	El sistema impide la operación indicando que la baja por caducidad o merma es facultad exclusiva del rol `Administrador` (RF04, RF15).	

UT-06. Búsqueda multicriterio de insumos en almacén (Nombre, Categoría y Fecha de Caducidad)

> **Trazabilidad:** UC0 | TUC-10 | RF05, RNF07 | MET-01, MET-12 | Riesgo: R1 | UI: `InventarioView.xaml` | **Escenario:** Positivo / Negativo | **Prioridad:** Media | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	En la barra de búsqueda de `InventarioView`, filtrar por el texto parcial de nombre `"Leche"`.	El `DataGrid` de WPF se actualiza en menos de 2 segundos mostrando únicamente los lotes correspondientes a `INS-01 (Leche Entera Pasteurizada 1L)`: `LOT-501`, `LOT-502` y `LOT-503`.	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	Limpiar el filtro de nombre y seleccionar en el filtro de Categoría el valor `"Frutas y Verduras"`.	La tabla muestra exclusivamente los lotes pertenecientes a dicha categoría (ej. `INS-02 Manzana Roja Fresca Kg`).	
3	Aplicar el filtro por rango o fecha de caducidad específica (próximos 7 días).	El sistema filtra y despliega únicamente `LOT-501` (+2 días) y `LOT-502` (+6 días), excluyendo `LOT-503` (+25 días).	
4	Ingresar un criterio de búsqueda inexistente en el almacén (ej. `"ProductoInexistenteXYZ"`).	La tabla se muestra vacía con un mensaje informativo de *"Sin coincidencias encontradas"* sin arrojar excepciones no controladas en .NET.	

UT-07. Consulta de urgencia semafórica de lotes y simulación de fechas

> **Trazabilidad:** UC0 (Paso 4) | TUC-11 | RF06 Corregido, RNF01 | MET-07, MET-08 | Riesgo: R2 | UI: `DashboardView.xaml`, `InventarioView.xaml`, `Simulación` | **Escenario:** Positivo / Valores Límite | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Ingresar a `DashboardView` e `InventarioView` y verificar la coloración y ordenamiento de urgencia de los lotes de `INS-01`.	El sistema muestra el código de colores semafórico en WPF: `LOT-501` (+2 días) resaltado en **Rojo (Crítico)** al inicio de la lista de prioridad, `LOT-502` (+6 días) en **Amarillo (Preventivo)** y `LOT-503` (+25 días) en **Verde (Óptimo)**.	
2	Desde `DashboardView`, hacer clic en el botón `Simular Fecha` para abrir el estado `Simulación` (`MainWindow - VOLUNTARIO - SIMULACION FECHA`) y adelantar la fecha del reloj simulado en `+5 días`.	El sistema recalcula dinámicamente la urgencia: `LOT-501` pasa a estado vencido/bloqueado para entrega, y `LOT-502` (al que ahora le resta 1 día) cambia automáticamente de Amarillo a **Rojo (Crítico)** actualizando el contador de `Productos por caducar`.	
3	Hacer clic en `OK` dentro de la vista `Simulación` para restablecer la fecha actual de operación y volver a `DashboardView`.	El sistema restaura la fecha real sin alterar los timestamps permanentes almacenados en Oracle Database (Postcondición 4).	

UT-08. Registro de nuevos donantes, validación de duplicados y vinculación al historial

> **Trazabilidad:** UC1.1 (Basic Path, 1a, 2a, 3a) | UC0 (Paso 2) | TUC-04, TUC-05 | RF07, RF09 | MET-11, MET-12 | Riesgo: R8 | UI: `DashboardView.xaml`, `LogisticaView.xaml` | **Escenario:** Positivo / Negativo | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Administrador`, abrir el formulario de alta de donantes y capturar: `nombreOrganización = "Supermercados del Valle S.A."`, `telefono = "5554443322"`, `correo = "rsc@supervalle.com"`.	El sistema verifica que no sea un registro duplicado (UC1.1 Paso 2), persiste al donante en la tabla `Donante` en Oracle mediante Dapper y crea su nodo correspondiente en Neo4j 5 (Postcondición 2).	
2	Registrar un nuevo lote entrante de alimento y vincularlo al perfil de `"Supermercados del Valle S.A."`; posteriormente, consultar el historial de aportaciones de dicho donante.	La relación 1 a * entre `Donante` y `Lote` queda establecida tanto en Oracle como en Neo4j; el perfil del donante lista el lote aportado con su fecha, cantidad y estado actual.	
3	Intentar guardar un donante duplicado (UC1.1 Alterno 2a), con datos inválidos (1a) u omitiendo campos obligatorios (3a).	El sistema detecta y rechaza el duplicado o los campos inválidos/vacíos, notificando al Administrador sin duplicar filas en Oracle.	

UT-09. Registro de beneficiarios y validación del número de dependientes en WPF

> **Trazabilidad:** UC1.5 | UC0 | TUC-12 | RF08, Nuevo RF14 | MET-10, MET-12 | Riesgo: R3 | UI: `LogisticaView.xaml` | **Escenario:** Positivo / Negativo | **Prioridad:** Media | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Voluntario`, abrir el registro de beneficiarios y capturar: `nombreRepresentante = "Elena Morales Díaz"` y `numDependientes = 4`.	El registro se almacena exitosamente en la tabla `Beneficiario` en Oracle vía Dapper y se crea su nodo receptor en Neo4j 5.	
2	Intentar capturar en el campo `numDependientes` letras o símbolos (ej. `"Cuatro"` o `"-2"`).	El validador numérico de WPF (Nuevo RF14) resalta la casilla en rojo y desactiva el botón de guardado.	

UT-10. Registro de salida de recursos guiado por el motor FIFO, Observer y bloqueo por violación FIFO

> **Trazabilidad:** UC1.5 (Basic Path, 1a, 2a) | UC0 (Paso 5, Flujo Alterno 4A) | TUC-13, TUC-14 | RF10 | MET-08, MET-12 | Riesgo: R2, R9 | UI: `LogisticaView.xaml`, `InventarioView.xaml` | **Escenario:** Positivo / Negativo | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	En el módulo de salidas/entregas, seleccionar el insumo `INS-01 (Leche Entera 1L)` y al beneficiario `BEN-201`, pero intentar seleccionar manualmente el lote nuevo `LOT-503` (+25 días) o `LOT-502` (+6 días) existiendo saldo disponible en el lote más antiguo `LOT-501` (+2 días, 50 u.) — Flujo Alterno 4A.	El motor FIFO **cancela inmediatamente la entrega manual**, despliega una alerta de *"Violación de regla FIFO: Existe un lote con caducidad más próxima"* y **autoselecciona obligatoriamente el lote `LOT-501`**.	
2	Confirmar la salida de `30 unidades` del lote autoseleccionado `LOT-501` asignándola al beneficiario `BEN-201`.	Se registra la salida en `Entrega_Logistica` con `cantidadEntregada = 30`; el stock de `LOT-501` en Oracle se descuenta de `50` a `20` unidades, el método `Observer` envía la notificación de actualización (`UC1.5`) y se mapea la entrega en Neo4j 5 (Postcondición 5).	
3	Intentar registrar una salida por `25 unidades` sobre `LOT-501` cuando su saldo restante es de `20 unidades` o sobre un insumo sin donaciones (`UC1.5 Alterno 1a`).	El sistema valida la disponibilidad real del lote, impide dejar stock negativo y solicita ajustar la cantidad al saldo del lote o dividir la entrega con el siguiente lote en orden FIFO (`LOT-502`).	

UT-11. Restauración de insumos al inventario por entregas canceladas o devoluciones

> **Trazabilidad:** UC1.5 | UC0 | TUC-15 | Nuevo RF16, RF10 | MET-08, MET-12 | Riesgo: R9, R11 | UI: `LogisticaView.xaml`, `InventarioView.xaml` | **Escenario:** Positivo / Negativo | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Voluntario`, seleccionar la entrega recién realizada de `30 unidades` del lote `LOT-501` y registrar su cancelación/devolución por parte del beneficiario.	El sistema reintegra las `30 unidades` al lote `LOT-501` en Oracle Database (restaurando su stock a `50 unidades`) y actualiza el estado de la entrega en Neo4j 5.	
2	Consultar la tabla en `InventarioView` y la cola de despacho del motor FIFO tras la restauración.	El lote `LOT-501` recupera exactamente su **posición original de prioridad número 1** en la cola FIFO según su fecha de caducidad original (+2 días), sin ser tratado como un lote nuevo.	
3	Intentar restaurar al inventario activo una devolución cuyo lote ya superó su fecha de caducidad durante el traslado.	El sistema impide su reincorporación como stock distribuible y exige canalizarlo al flujo de baja por merma (RF04/RF15).	

UT-12. Consulta de stock agrupado por categorías y exportación de reportes en PDF (QuestPDF) y CSV

> **Trazabilidad:** UC1.4 (Basic Path, 1a, 2a, 3a) | UC0 (Paso 6) | TUC-16, TUC-17 | RF11, RF12, RNF08 | MET-02, MET-12 | Riesgo: R1 | UI: `DashboardView.xaml`, `InventarioView.xaml`, `Exito CSV` | **Escenario:** Positivo / Negativo | **Prioridad:** Alta | **Ejecuta:** Desarrollador / Tester

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Con sesión de `Administrador`, ejecutar la consulta de total de stock disponible agrupado por categorías de alimentos (RF11, UC1.4).	El sistema ejecuta la consulta de agregación vía Dapper/GraphQL (`SUM(cantidad) GROUP BY categoria`) y muestra los totales exactos para `Lácteos (250 u.)`, `Frutas y Verduras`, `Granos y Cereales (95 u.)`, excluyendo mermas dadas de baja.	
2	En `InventarioView`, hacer clic en `Exportar CSV` (RF12).	El sistema escribe el archivo físico `.csv` usando `System.IO` en el disco duro local y despliega el modal `Exito` / `Exito CSV` mostrando el mensaje *"Reporte exportado dinamicamente! Guardado en: PATH"* con el botón `OK` (Postcondición 6).	
3	Seleccionar la opción de exportación del reporte histórico de inventario y logística en formato `PDF` mediante `QuestPDF` (RF12).	Se genera y guarda exitosamente el archivo `.pdf` maquetado con `QuestPDF` en la ruta local especificada, conteniendo el resumen de existencias, entregas logísticas y mermas.	

PRUEBAS INTEGRALES (IT)

IT-01. Flujo integral de Abastecimiento: Alta de Donante, ingreso de Lote por Fachada y sincronización Oracle–Neo4j

> **Trazabilidad:** UC0 (Pasos 1, 2, 3) | UC1.1, UC1.2, UC1.3 | TUC-04, TUC-06 | RF01, RF02, RF07, RF09, Nuevo RF14, RNF11, RNF12, RNF13 | MET-11, MET-13 | Riesgo: R6, R8 | UI: `LoginView`, `DashboardView`, `InventarioView`, `LogisticaView` | **Escenario:** Positivo / Integración Políglota | **Prioridad:** Alta | **Ejecuta:** Tester / Administrador / Voluntario

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	El `Administrador` inicia sesión en WPF, registra al donante `DON-102 (Central de Abastos CDMX)` y verifica su creación tanto en el catálogo relacional como en la vista logística.	El registro queda persistido en la tabla `Donante` de Oracle Database 21c XE vía Dapper y se inicializa el nodo `(:Donante {idDonante: 102})` en Neo4j 5 vía `Neo4j.Driver`.	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	El `Voluntario` inicia sesión y registra mediante la Fachada el ingreso físico de dos lotes de `INS-02 (Manzana Roja Kg)` aportados por `DON-102`: `LOT-701` (caduca en 4 días) y `LOT-702` (caduca en 18 días).	Ambos lotes se almacenan en Oracle bajo transacción ACID vinculados a `idDonante = 102` e `idInsumo = 2`, crean su relación con el donante en Neo4j y aparecen coloreados en Amarillo y Verde respectivamente en `InventarioView`.	
3	Consultar el perfil histórico de `DON-102` y el contador `Donaciones Hoy` en `DashboardView` (y mediante consulta GraphQL en Hot Chocolate).	El panel principal incrementa en tiempo real el indicador de donaciones del día y el historial del donante refleja ambos lotes sin discrepancias entre vistas.	

IT-02. Flujo integral de Distribución FIFO, Notificación Observer y mapeo de red logística en Neo4j 5

> **Trazabilidad:** UC0 (Pasos 4, 5) | UC1.5 | TUC-11, TUC-12, TUC-13 | RF06, RF08, RF10, RNF01, RNF11, RNF13 | MET-08, MET-11, MET-13 | Riesgo: R2, R6 | UI: `DashboardView`, `InventarioView`, `LogisticaView` | **Escenario:** Positivo / Negativo / Integración Políglota | **Prioridad:** Alta | **Ejecuta:** Tester / Voluntario

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	El `Voluntario` consulta el semáforo en `InventarioView`, identifica el lote prioritario de `Manzana Roja Kg` (`LOT-701`, +4 días) y navega a `LogisticaView (Neo4j)`.	La transición de estado desde `DashboardView` hacia `LogisticaView` carga el motor de grafos (`WebView2` / `Neo4j.Driver`) y los lotes disponibles ordenados por fecha de caducidad ascendente.	
2	Registrar la entrega de `15 Kg` de `Manzana Roja` para la beneficiaria `BEN-201 (María Guadalupe Hernández)`.	El sistema fuerza la selección de `LOT-701` (FIFO), descuenta `15 Kg` en Oracle Database, inserta el registro en `Entrega_Logistica` y el método `Observer` emite la notificación de cambio.	
3	Ejecutar en `LogisticaView` la consulta Cypher en Neo4j 5 para visualizar la ruta de distribución entre el donante origen y el beneficiario final (Postcondición 5 y RNF13).	Neo4j presenta el grafo actualizado conectando los nodos `(:Donante {id: 102}) -[:APORTO]-> (:Lote {id: 701}) -[:ENTREGADO_A]-> (:Beneficiario {id: 201})`, permitiendo optimizar la trazabilidad de extremo a extremo.	

IT-03. Flujo integral de Excepciones Logísticas: Devolución de entrega, reingreso FIFO y baja por Merma

> **Trazabilidad:** UC0 | UC1.4, UC1.5 | TUC-09, TUC-15, TUC-16 | RF03, RF04, RF09, Nuevo RF15, Nuevo RF16 | MET-08, MET-11 | Riesgo: R7, R8, R9 | UI: `LogisticaView`, `InventarioView`, `Gestión de Lote` | **Escenario:** Positivo / Negativo / Integración | **Prioridad:** Alta | **Ejecuta:** Tester / Voluntario / Administrador

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	El `Voluntario` registra la cancelación parcial/total de la entrega de `LOT-701` en `LogisticaView` (Nuevo RF16).	El inventario en Oracle recupera las unidades devueltas en `LOT-701` conservando su fecha de caducidad original (+4 días) por encima de `LOT-702` (+18 días).	
2	Al detectarse que `5 Kg` de la devolución sufrieron daño físico en transporte, el `Administrador` ingresa a `Gestión de Lote` y da de baja esos `5 Kg` clasificando el motivo `"Daño en retorno logístico"` (RF04, Nuevo RF15).	Se descuentan los `5 Kg` dañados del stock activo, se crea el registro en la tabla `Merma` y se mantiene el vínculo de trazabilidad hacia el donante `DON-102`.	
3	El `Administrador` consulta el stock por categoría (RF11) y exporta el reporte CSV y PDF con `QuestPDF` (RF12).	Tanto la consulta en pantalla como los archivos CSV/PDF exportados reflejan exactamente el saldo neto tras la entrega, la restauración y la merma.	

IT-04. Flujo integral de Navegación en la Máquina de Estados de Interfaz Gráfica (WPF XAML)

> **Trazabilidad:** Diagrama de Máquina de Estados y Wireframes UI | RNF02, RNF04, RNF10 | MET-06, MET-09, MET-13 | Riesgo: R12 | UI: Todas las vistas WPF | **Escenario:** Positivo / Integración UI-Fachada | **Prioridad:** Media | **Ejecuta:** Tester / Robot

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Recorrer las transiciones desde `Initial` → `[Abrir App WPF]` → `LoginView` → `[Ingresar credenciales incorrectas]` → `Error de Autenticación` → `[Click en OK]` → `LoginView` → `[Ingresar credenciales correctas]` → `DashboardView`.	Todas las ventanas XAML y diálogos modales de `MaterialDesignThemes` abren y cierran limpiamente sin dejar subprocesos bloqueados ni ventanas duplicadas.	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	Desde `DashboardView`, ejecutar las transiciones de ida y vuelta hacia `Registro Exitoso` (`[Click en "Registrar Ingreso"]` / `[Click en OK]`), `LogisticaView` (`[Click en "Logística"]` / `[Click en "Panel de Control"]`) y `Simulación` (`[Click en "Simular Fecha"]` / `[Click en OK]`).	Cada botón de la barra lateral y superior invoca al controlador/Fachada correspondiente y refresca los indicadores del `DashboardView` al retornar.	
3	Desde `DashboardView`, transicionar a `InventarioView` (`[Click en "Inventario"]`), abrir y regresar de `Gestión de Lote` (`[Click en Editar / Eliminar]` / `[Confirmar Acción]`), abrir y regresar de `Éxito CSV` (`[Click en "Exportar CSV"]` / `[Click en OK]`), volver a `DashboardView` y pulsar `[Click en "Cerrar Sesión"]`.	El ciclo completo de la máquina de estados concluye en `LoginView` cerrando de forma segura las sesiones de usuario y liberando recursos gráficos en .NET 10 (Postcondición 1 y 7).	

IT-05. Consistencia Transaccional Políglota (Rollback coordinado entre Oracle 21c XE y Neo4j 5 en Docker)

> **Trazabilidad:** UC0 (Flujos Alternos 2A y 5A) | TUC-07, TUC-18 | RNF06, RNF11, RNF12, RNF13 | MET-05, MET-13 | Riesgo: R4, R6 | UI: `LogisticaView`, Capa Fachada | **Escenario:** Negativo / Integración Políglota | **Prioridad:** Alta | **Ejecuta:** Tester / Desarrollador

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Pausar temporalmente el contenedor Docker de Neo4j 5 justo en el instante en que el `Voluntario` confirma una salida de inventario en `LogisticaView`.	La Fachada en C# captura la excepción de `Neo4j.Driver`, ejecuta `Rollback()` sobre la transacción de `OracleConnection` en Dapper y muestra alerta de error transaccional en WPF.	
2	Verificar las existencias del lote en Oracle Database 21c XE y los nodos de entrega en Neo4j tras el fallo simulado.	Cero inconsistencias: el stock del lote en Oracle conserva su cantidad íntegra previa y en Neo4j no quedan nodos de entrega huérfanos.	
3	Reanudar el contenedor Docker de Neo4j 5 y repetir el registro de la entrega logística.	La transacción se confirma exitosamente en ambos motores (Oracle y Neo4j) en menos de 2 segundos.	

IT-06. Flujo Integral de Negocio de Nivel 0 (Extremo a Extremo: Caso de Uso Principal "Gestionar Banco de Alimentos")

> **Trazabilidad:** UC0 (Basic Path Pasos 1 al 7 completos) | UC1.1 a UC1.5 | TUC-01 a TUC-18 | RF01 a Nuevo RF16 | MET-01 a MET-13 | UI: Todas las pantallas WPF | **Escenario:** Positivo / Integración E2E | **Prioridad:** Alta | **Ejecuta:** Tester / Todos los Actores / Robot

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	**Inicio de Sesión (Paso 1 Basic Path - UC1.2):** El Actor (`Administrador` / `Voluntario`) ingresa sus credenciales válidas en `LoginView.xaml` y el sistema las autentica.	Sesión iniciada correctamente con permisos de rol cargados y conexiones activas a Oracle 21c XE y Neo4j 5.	
2	**Registro de Donante (Paso 2 Basic Path - UC1.1):** El `Administrador` registra los datos de un nuevo donante y vincula su información de contacto en el sistema.	Perfil del donante persistido de forma segura en Oracle y Neo4j, listo para asociarle donaciones (Postcondición 2).	
3	**Ingreso Físico de Perecederos (Paso 3 Basic Path - UC1.3):** El `Voluntario` registra el ingreso físico de los recursos perecederos capturando nombre, categoría, lote, cantidad y fecha de caducidad válida.	Existencias guardadas y actualizadas en Oracle Database garantizando integridad ACID y relación con el donante en Neo4j (Postcondición 3).	
4	**Consulta Semafórica (Paso 4 Basic Path):** El `Voluntario` consulta el estado del inventario en WPF y visualiza el código de colores semafórico que le indica qué productos están en estado crítico o preventivo.	Interfaz gráfica actualizada mostrando las alertas de caducidad correspondientes sin alterar los datos subyacentes (Postcondición 4).	
5	**Salida Guiada por FIFO (Paso 5 Basic Path - UC1.5):** El `Voluntario` registra la salida de recursos para un beneficiario, siendo forzado por el sistema a seleccionar los lotes más antiguos siguiendo la regla FIFO.	Inventario descontado en Oracle respetando FIFO, aviso Observer emitido y ruta de entrega entre donante y beneficiario mapeada como nuevo nodo/relación en Neo4j (Postcondición 5).	
6	**Consulta de Stock y Exportación (Paso 6 Basic Path - UC1.4):** El `Administrador` consulta el total de stock disponible por categoría y exporta un reporte logístico en formato PDF (`QuestPDF`) y CSV.	Archivos físicos en formato PDF y CSV generados y guardados exitosamente en el disco duro local en ≤ 5 segundos (Postcondición 6).	
7	**Cierre de Sesión Seguro (Paso 7 Basic Path):** El Actor finaliza sus tareas y presiona `Cerrar Sesión` en el sistema.	La sesión se cierra de manera segura, cerrando las conexiones a las bases de datos y liberando la memoria RAM consumida en .NET 10 (Postcondiciones 1 y 7).	

PRUEBAS DE SISTEMA (ST — RENDIMIENTO, RECURSOS, CONFIABILIDAD Y USABILIDAD)

ST-01. Rendimiento en búsquedas filtradas con Dapper (≤ 2 s) y exportación de reportes con 10,000 registros (≤ 5 s)

> **Trazabilidad:** UC0 | UC1.4 | TUC-10, TUC-16 | RF05, RF11, RF12, RNF07, RNF08 | MET-01, MET-02 | Riesgo: R1 | UI: `InventarioView.xaml` | **Escenario:** Rendimiento / Volumen | **Prioridad:** Alta | **Ejecuta:** QA Lead / Tester Automation

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Cargar en los contenedores Docker de Oracle Database 21c XE y Neo4j 5 el lote de prueba de volumen `DATASET-10K` (10,000 registros distribuidos en lotes, insumos y entregas logísticas).	Base de datos poblada con 10,000 registros activos e índices primarios/secundarios operativos.	
2	Ejecutar en `InventarioView` 10 búsquedas filtradas consecutivas combinando nombre, categoría y fecha de caducidad, midiendo con `System.Diagnostics.Stopwatch` el tiempo `T_respuesta = T_renderizado - T_solicitud` (RNF07).	El 100% de las búsquedas filtradas con Dapper procesa y renderiza los resultados en el `DataGrid` de WPF en un tiempo **≤ 2.0 segundos** (cumpliendo MET-01).	
3	Solicitar la generación y guardado en disco del reporte completo de los 10,000 registros tanto en formato **CSV** como en formato **PDF** con `QuestPDF` de manera asíncrona (`System.Threading.Tasks` — RNF08).	Ambos archivos se generan y escriben íntegramente en el disco local en un tiempo **≤ 5.0 segundos** sin bloquear el hilo de interfaz gráfica (UI Dispatcher Thread) de WPF (cumpliendo MET-02).	

ST-02. Monitoreo de consumo de memoria RAM (≤ 512 MB) bajo operación continua en .NET 10 y WPF

> **Trazabilidad:** UC0 | TUC-18 | RNF09, RNF10 | MET-03 | Riesgo: R5 | UI: Todas las pantallas WPF | **Escenario:** Rendimiento / Recursos Locales | **Prioridad:** Alta | **Ejecuta:** QA Lead / Desarrollador

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Conectar las herramientas de diagnóstico de Visual Studio (*.NET Memory Profiler / Diagnostic Tools / Task Manager*) al proceso `SGRC.exe` (.NET 10 CLR) con el dataset de 10,000 registros cargado.	El consumo inicial de memoria RAM del proceso WPF tras cargar `DashboardView` se mantiene por debajo de los 250 MB.	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	Ejecutar durante 20 minutos continuos cambios de vista XAML, consultas semafóricas, simulaciones de fecha, visualización de grafos en `WebView2` / Neo4j y 5 exportaciones sucesivas de reportes PDF/CSV.	La aplicación opera fluidamente consumiendo en todo momento un **máximo ≤ 512 MB de memoria RAM** (cumpliendo RNF09 y MET-03), sin presentar fugas de memoria (*Memory Leaks*) ni excepciones `OutOfMemoryException`.	
3	Cerrar sesión desde la interfaz (Paso 7 del Caso de Uso).	Las conexiones de `Oracle.ManagedDataAccess.Core` y `Neo4j.Driver` hacen `Dispose()` y el Garbage Collector de .NET 10 libera los objetos de sesión (Postcondición 7).	

ST-03. Confiabilidad: Respaldo local automático cada 24h y persistencia ACID ante cierre inesperado

> **Trazabilidad:** UC0 (Flujo Alterno 5A) | TUC-18 | RNF05, RNF06, RNF12 | MET-04, MET-05 | Riesgo: R4 | UI: `InventarioView.xaml`, Contenedor Oracle 21c XE | **Escenario:** Confiabilidad / Recuperación ante Desastres | **Prioridad:** Alta | **Ejecuta:** QA Lead / DBA

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Verificar la tarea programada de copia de seguridad local (RNF05) y disparar el ciclo de respaldo automático de 24 horas.	El sistema genera exitosamente el archivo de respaldo local de la base de datos en el directorio de seguridad con timestamp del día y verificación de integridad al 100% (MET-04).	
2	Confirmar el registro del lote `LOT-801`; inmediatamente después, iniciar una segunda transacción de lote (`LOT-802`) y antes de su confirmación forzar el cierre abrupto del proceso `SGRC.exe` (`taskkill /F` simulando apagón — Flujo Alterno 5A).	Oracle Database 21c XE detecta la interrupción abrupta de la sesión y cancela automáticamente la transacción incompleta de `LOT-802`.	
3	Reiniciar la aplicación WPF del SGRC, iniciar sesión y consultar `InventarioView` (RNF06, RNF12).	El sistema recupera el último estado válido garantizando el 100% de las propiedades ACID: el lote confirmado `LOT-801` permanece intacto y no existen registros corruptos ni bloqueos de tabla por `LOT-802` (MET-05).	

ST-04. Auditoría de Usabilidad y Ergonomía Visual en WPF: Regla de máximo 3 clics, fuentes ≥ 14pt y modales

> **Trazabilidad:** UC0 | TUC-06, TUC-11 | RNF01, RNF02, RNF03, RNF04 | MET-06, MET-07, MET-09 | Riesgo: R12, R13 | UI: Todas las pantallas WPF (`MaterialDesignThemes 5.3.1`) | **Escenario:** Usabilidad / Servicio | **Prioridad:** Media | **Ejecuta:** Tester QA / Voluntario UAT

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Situarse en el menú principal (`DashboardView`) y contar el número de clics necesarios para llegar y abrir el formulario de registro de inventario (RNF02).	El usuario accede al formulario en **1 a 2 clics** (directamente desde el botón `Registrar Ingreso` en `DashboardView` o vía `Inventario` → `Registrar`), cumpliendo holgadamente el umbral **≤ 3 clics** (MET-06).	
2	Inspeccionar los recursos XAML (`FontSize` en estilos de `MaterialDesignThemes`) de las etiquetas, tablas y botones en todas las pantallas de WPF (RNF03).	El 100% de los textos operativos utiliza un tamaño tipográfico **mínimo de 14pt**, garantizando legibilidad a distancia en el piso de la bodega (MET-07).	
3	Solicitar a un voluntario de nuevo ingreso realizar un alta de insumo y una entrega FIFO guiándose únicamente por la interfaz WPF y el semáforo de colores.	El voluntario completa ambas tareas sin errores ni asistencia externa en menos de 3 minutos (mitigando el riesgo R12 de alta rotación de personal).	

ST-05. Auditoría de Arquitectura MVC / Fachada, GraphQL, Persistencia Políglota y Seguridad de Datos

> **Trazabilidad:** UC0 | RNF10, RNF11, RNF12, RNF13 | MET-13 | Riesgo: R6, R10, R14 | UI: Solución C# (.NET 10), Hot Chocolate GraphQL y Bases de Datos | **Escenario:** Seguridad / Soportabilidad | **Prioridad:** Alta | **Ejecuta:** Arquitecto SW / QA Lead

Paso	Actividad	Resultado Esperado	Fecha y Hora
1	Inspeccionar la estructura de proyectos y namespaces en la solución C# (.NET 10) verificando la separación entre las capas `Model` , `View (WPF XAML)` , `Controller / Fachada` y el servidor `SGRC.GraphQL.Api` con `Hot Chocolate 16.7.5` (RNF10).	El 100% del código respeta el desacoplamiento arquitectónico con 0 consultas SQL o Cypher incrustadas directamente en el code-behind de las vistas XAML (MET-13).	

Paso	Actividad	Resultado Esperado	Fecha y Hora
2	Intentar inyectar cadenas con caracteres de escape SQL (ej. `' OR '1'='1'; DROP TABLE Lote; --`) en el campo `txtUser` de `LoginView` y en el buscador de `InventarioView`.	El uso de parámetros tipados en `Dapper` (`@Param`), `Oracle.ManagedDataAccess.Core` y parámetros en `Neo4j.Driver` neutraliza el vector de inyección tratándolo como literal de texto inofensivo.	

PRUEBAS DE REGRESIÓN Y ROBOT AUTOMATIZADO (RT)

RT-01. Ejecución del Robot Automatizado de Pruebas en C# (FlaUI / xUnit) sobre WPF — Escenario Principal: Registro Exitoso de Lote Perecedero y Flujo Integral

> **Trazabilidad:** Rúbrica Punto de Pruebas (Plan, Test Use Case, Guía y Robot que la siga) | Escenario Principal acordado: *Registro exitoso de un lote de alimento (WPF → Validaciones → Fachada → Oracle 21c XE → Relación Donante en Neo4j 5)* | UC0 y UC1.3 | TUC-01, TUC-06, TUC-07 | **Escenario:** Automatización UI en C# / Regresión | **Prioridad:** Crítica | **Ejecuta:** Robot Automatizado en C# (`FlaUI.UIA3` / `xUnit`) / QA Lead

Paso	Actividad Ejecutada Automáticamente por el Robot en WPF	Resultado Esperado Verificado por el Robot	Fecha y Hora
1	**Arranque del Ejecutable y Autenticación en WPF (RF01):** El Robot en C# lanza el proceso del ejecutable WPF (`SGRC.exe`), localiza por `AutomationId / x:Name` los controles de `LoginView` (o verifica acceso al panel principal), introduce las credenciales válidas (`txtUser = "voluntario1"` / `"admin"`, `txtPwd = "Vol2026!"` / `"Admin2026!"`), invoca el clic en `ENTRAR AL SISTEMA` y cierra el modal `Autenticacion Exitosa`.	El Robot verifica que la ventana principal `DashboardView` de WPF se encuentra activa en pantalla y lista para recibir interacciones en menos de 2 segundos.	
2	**Navegación en ≤ 3 clics al Formulario de Registro de Lote (RNF02):** El Robot localiza en `DashboardView` el botón `Registrar Ingreso` (o navega a `InventarioView`) y ejecuta el clic automático.	Se despliega el formulario WPF de captura de nuevo lote perecedero cumpliendo la métrica MET-06 (≤ 3 clics).	
3	**Validación Preventiva en WPF (Nuevo RF14 — Opcional/Pre-chequeo):** El Robot verifica que los controles de entrada están habilitados y que las reglas de validación impiden guardar datos corruptos antes de inyectar el lote válido.	Los controles XAML responden adecuadamente al árbol de automatización UIA3 de Windows.	

Paso	Actividad Ejecutada Automáticamente por el Robot en WPF	Resultado Esperado Verificado por el Robot	Fecha y Hora
4	**Captura Automática del Nuevo Lote de Alimento (RF02, RF07, RF09):** El Robot llena automáticamente cada campo del formulario: Nombre del insumo (`"Leche Entera Pasteurizada 1L"`), Categoría (`"Lácteos"`), Código de Lote (`"LOT-ROBOT-01"`), Cantidad numérica válida (`60`), Fecha de caducidad futura válida (`Fecha_Actual + 30 días`) y Donante asociado (`DON-101 Fundación Alimentar México A.C.`), y presiona el botón `Guardar / Confirmar` .	La petición pasa de la vista WPF a la **Fachada**, ejecuta el `INSERT` transaccional ACID en **Oracle Database 21c XE** mediante `Dapper` y crea la relación del lote con el nodo del donante en **Neo4j 5** mediante `Neo4j.Driver` .	
5	**Verificación de Confirmación y Visualización en Inventario (Postcondición):** El Robot detecta el diálogo/mensaje de confirmación (`Registro Exitoso`), hace clic en `OK` y verifica en el `DataGrid` de `InventarioView` que el lote `LOT-ROBOT-01` aparece listado con `60 unidades` y semáforo en color **Verde (Óptimo)**.	El sistema registra el lote correctamente, muestra la confirmación y permite visualizarlo en la tabla de inventario con 0 errores de ejecución.	

RT-02. Regresión Incremental por Épica del Tablero Kanban (5 Épicas del SGRC)

> **Trazabilidad:** Tablero Kanban SGRC (5 Épicas en columna Done) y Control de Versiones | **Prioridad:** Alta | **Ejecuta:** QA Lead / Desarrolladores

Paso	Actividad por Épica del Proyecto	Resultado Esperado	Fecha y Hora
1	**Épica 1 — Arquitectura e Infraestructura (MVC/Fachada + Persistencia Políglota Oracle 21c XE / Neo4j 5 en Docker + GraphQL):** Ejecutar `IT-05` , `ST-02` , `ST-03` y `ST-05` .	Aprobados al 100%. Capas desacopladas en .NET 10 y conectores Dapper/Oracle y Neo4j.Driver operando sin fugas de memoria.	
2	**Épica 2 — Seguridad y Acceso (Módulo de Autenticación RF01 + CRUD de Usuarios RF13):** Ejecutar `UT-01` , `UT-02` + regresión de Épica 1.	Aprobados al 100%. Validación de sesiones diferenciadas para Administrador y Voluntario, bloqueo al 3.er intento y CRUD con modal de confirmación (RNF04).	

Paso	Actividad por Épica del Proyecto	Resultado Esperado	Fecha y Hora
3	**Épica 3 — Gestión de Inventario y Lógica (Motor FIFO, Registro/Validación RF02, RF03, RF04, RF05, RF06, RF14, RF15 y Semáforo RNF01):** Ejecutar `UT-03`, `UT-04`, `UT-05`, `UT-06`, `UT-07`, `ST-04` + regresión de Épicas 1–2.	Aprobados al 100%. Bloqueo total de fechas vencidas y letras en campos numéricos en WPF, semáforo operativo y trazabilidad de mermas.	
4	**Épica 4 — Entidades y Logística (Donantes RF07/RF09, Beneficiarios RF08, Salida FIFO RF10 y Devoluciones RF16):** Ejecutar `UT-08`, `UT-09`, `UT-10`, `UT-11`, `IT-01`, `IT-02`, `IT-03` + regresión de Épicas 1–3.	Aprobados al 100%. Entregas guiadas obligatoriamente por FIFO, método Observer activo, restauración de devoluciones en su orden original y mapeo en Neo4j 5.	
5	**Épica 5 — Salidas, Reportes y Cierre Integral (Stock por Categoría RF11, Exportación QuestPDF/CSV RF12, Rendimiento RNF07/RNF08 y Robot):** Ejecutar `UT-12`, `IT-04`, `IT-06`, `ST-01` y `RT-01 (Robot Automatizado)`.	Aprobados al 100%. Los 16 RF y 13 RNF cumplen el 100% de los criterios de aceptación para liberación final.	

7. MATRIZ DE TRAZABILIDAD

7.1 Casos de Uso (Nivel 0 UC0 y Nivel 1 UC1.1 a UC1.5) y Flujos Alternos vs. Scripts y Fichas TUC

ID Caso de Uso / Flujo	Nombre del Caso de Uso / Escenario en Enterprise Architect	Scripts (UT, IT, ST, RT) y Fichas TUC Asociadas
UC0 (Basic Path)	Gestionar Banco de Alimentos — Flujo Básico Completo (Pasos 1 al 7: Login, Alta Donante, Ingreso Perecedero, Consulta Semafórica, Salida FIFO, Reporte PDF/CSV y Cierre de Sesión)	UT-01 a UT-12, IT-01 a IT-06, ST-01 a ST-05, RT-01, RT-02 | **TUC-01 a TUC-18**
UC0 — Alterno 1A	Autenticación_Fallida (Credenciales inválidas deniegan acceso; al 3.er error consecutivo la cuenta se bloquea temporalmente)	UT-01 (Pasos 3 y 4), IT-04 | **TUC-02**
UC0 — Alterno 2A	Caducidad_Invalida (Ingresar una fecha vencida aborta el registro con Rollback en Oracle y lanza alerta de "Rechazo automático")	UT-03 (Paso 2), UT-04 (Paso 3), IT-05 | **TUC-07 (Paso 1)**
UC0 — Alterno 3A	Error_Captura (Escribir letras en campos numéricos desactiva el botón de guardado en WPF y resalta la casilla en rojo)	UT-03 (Paso 3), UT-04 (Paso 3), UT-09 (Paso 2) | **TUC-07 (Paso 2), TUC-12**
UC0 — Alterno 4A	Violación_Fifo (Intentar sacar un lote nuevo existiendo uno más antiguo cancela la entrega, alerta y autoselecciona el lote correcto)	UT-10 (Paso 1), IT-02 (Paso 2) | **TUC-14 (Paso 1)**

ID Caso de Uso / Flujo	Nombre del Caso de Uso / Escenario en Enterprise Architect	Scripts (UT, IT, ST, RT) y Fichas TUC Asociadas
UC0 — Alterno 5A	Cierre_Inesperado (Apagón o cierre abrupto cancela transacciones incompletas; al reiniciar recupera el último estado válido ACID)	IT-05 (Pasos 1 y 2), ST-03 (Pasos 2 y 3) | **TUC-18 (Paso 2)**
UC1.1 (Nivel 1)	Gestionar donantes (Basic Path + Alternos 1a datos inválidos, 2a registro duplicado y 3a dato vacío)	UT-08, IT-01 | **TUC-04, TUC-05**
UC1.2 (Nivel 1)	Autenticar Usuario (Basic Path + Alternos 1a datos inválidos y 2a usuario/contraseña inexistente)	UT-01, IT-04, RT-01 (Paso 1) | **TUC-01, TUC-02**
UC1.3 (Nivel 1)	Registrar ingresos de insumos (Basic Path + Alternos 1a producto caducado y 2a dato faltante)	UT-03, UT-04, IT-01, RT-01 (Pasos 2–5) | **TUC-06, TUC-07, TUC-08**
UC1.4 (Nivel 1)	Generar reportes de stock (Basic Path + Alternos 1a filtro inválido, 2a dato inexistente y 3a sin registros)	UT-12, IT-03, ST-01 | **TUC-16, TUC-17**
UC1.5 (Nivel 1)	Procesar entrega logística (Basic Path con notificación Observer + Alternos 1a sin donaciones y 2a descarga de datos)	UT-09, UT-10, UT-11, IT-02, IT-03 | **TUC-12, TUC-13, TUC-14, TUC-15**

7.2 Requerimientos Funcionales (RF01 a Nuevo RF16) y No Funcionales (RNF01 a RNF13) vs. Scripts y TUC

ID Req.	Descripción del Requerimiento (Baseline SGRC)	Métrica / Riesgo	Scripts y TUC Asociados
RF01	Inicio de sesión de Voluntario y Administrador con usuario y contraseña válidos.	MET-12 / R10	UT-01, IT-01, IT-04, RT-01 | TUC-01, TUC-02
RF02	Registro de ingreso de inventario perecedero (nombre, categoría, lote, cantidad y caducidad).	MET-10 / R3	UT-03, IT-01, IT-06, RT-01 | TUC-06, TUC-07
RF03	Modificación de datos de un producto en el inventario por error de captura.	MET-12 / R3	UT-04, IT-04, RT-02 | TUC-08
RF04	Baja del inventario de productos que hayan caducado o se registren como mermas.	MET-09 / R7	UT-05, IT-03, RT-02 | TUC-09
RF05	Búsqueda de productos específicos filtrando por nombre, categoría o fecha de caducidad.	MET-01 / R1	UT-06, ST-01, RT-02 | TUC-10
RF06 Corr.	Consulta del nivel de urgencia de los lotes próximos a caducar para priorizar distribución.	MET-08 / R2	UT-07, IT-02, IT-06 | TUC-11, TUC-14
RF07	Registro de datos de nuevos donantes (nombre de organización/persona, teléfono y correo).	MET-11 / R8	UT-08, IT-01, IT-06 | TUC-04, TUC-05

ID Req.	Descripción del Requerimiento (Baseline SGRC)	Métrica / Riesgo	Scripts y TUC Asociados
RF08	Registro de beneficiarios anotando representante familiar y número de dependientes.	MET-12 / R3	UT-09, IT-02, RT-02 | TUC-12
RF09	Vinculación de cada donación entrante al perfil de un donante para historial de aportaciones.	MET-11 / R8	UT-08, IT-01, IT-03, RT-01 | TUC-04
RF10	Registro de salida de recursos guiado por el método FIFO asignándola a un beneficiario.	MET-08 / R2, R9	UT-10, IT-02, IT-06, RT-02 | TUC-13, TUC-14
RF11	Consulta del total de stock disponible agrupado por categorías de alimentos.	MET-12 / R1	UT-12, IT-03, IT-06 | TUC-16, TUC-17
RF12	Generación y exportación de reportes del historial de inventario y logística en PDF (QuestPDF) y CSV.	MET-02 / R1	UT-12, IT-03, IT-06, ST-01 | TUC-16, TUC-17
RF13	Creación, edición o eliminación de cuentas de acceso para el personal voluntario.	MET-09 / R10	UT-02, RT-02 | TUC-03
Nuevo RF14	Validación de datos de entrada impidiendo fechas de caducidad pasadas y letras en campos numéricos.	MET-10 / R3	UT-03, UT-04, UT-09, RT-01 | TUC-05, TUC-07, TUC-12
Nuevo RF15	Clasificación y registro del motivo específico de cada merma al dar de baja los insumos.	MET-11 / R7	UT-05, IT-03, RT-02 | TUC-09
Nuevo RF16	Restauración de insumos al inventario por entregas canceladas o devoluciones respetando posición FIFO.	MET-08 / R11	UT-11, IT-03, RT-02 | TUC-15
RNF01	Código de colores semafórico en tablas de inventario en WPF para indicar proximidad de caducidad.	MET-07 / R2	UT-07, IT-02, ST-04, RT-01 | TUC-11
RNF02	Navegación desde el menú principal hasta el formulario de registro de inventario en ≤ 3 clics.	MET-06 / R12	IT-04, ST-04, RT-01 | TUC-06
RNF03	Uso de fuentes tipográficas con tamaño mínimo de 14pt en XAML para asegurar legibilidad.	MET-07 / R13	ST-04 | TUC-11
RNF04	Despliegue de ventana emergente de confirmación antes de procesar la eliminación de cualquier registro.	MET-09 / R7, R10	UT-02, UT-05, IT-04, ST-04 | TUC-03, TUC-09
RNF05	Copia de seguridad local (respaldo) de la base de datos de forma automática cada 24 horas.	MET-04 / R4	ST-03 (Paso 1) | TUC-18
RNF06	Garantía de persistencia de los datos guardados en caso de cierre inesperado de la aplicación.	MET-05 / R4	IT-05, ST-03 (Pasos 2 y 3) | TUC-18
RNF07	Procesamiento y visualización de resultados de búsqueda filtrada con Dapper en tiempo ≤ 2 segundos.	MET-01 / R1	UT-06, ST-01 (Paso 2) | TUC-10

ID Req.	Descripción del Requerimiento (Baseline SGRC)	Métrica / Riesgo	Scripts y TUC Asociados
RNF08	Generación y guardado de reportes PDF (QuestPDF) / CSV en ≤ 5 segundos para hasta 10,000 registros.	MET-02 / R1	UT-12, ST-01 (Paso 3) | TUC-16
RNF09	Funcionamiento fluido en .NET 10 consumiendo un máximo de 512 MB de memoria RAM.	MET-03 / R5	ST-02 | TUC-18
RNF10	Estructuración del sistema siguiendo el patrón de arquitectura MVC / Fachada en C# (.NET 10) y WPF.	MET-13 / R14	IT-04, ST-02, ST-05, RT-02 | TUC-01, TUC-18
RNF11	Operación bajo arquitectura de persistencia políglota para manejo distribuido de información.	MET-13 / R6	IT-01, IT-02, IT-05, ST-05 | TUC-13
RNF12	Uso de Oracle Database 21c XE como motor relacional garantizando propiedades ACID.	MET-05 / R4, R6	UT-03, IT-01, IT-05, ST-03 | TUC-06, TUC-07, TUC-18
RNF13	Implementación de Neo4j 5 como motor de grafos para mapear y optimizar la red Donantes–Beneficiarios.	MET-13 / R6, R11	IT-01, IT-02, IT-05, RT-01 | TUC-04, TUC-12, TUC-13

PARTE II — ESPECIFICACIÓN DE CASOS DE PRUEBA (TEST USE CASES — FICHAS ESTRUCTURADAS TUC-01 A TUC-18)

ÍNDICE GENERAL DE TEST USE CASES (TUC-01 A TUC-18)

ID TUC	Caso de Uso Asociado (EA)	Req. / Escenario	Objetivo y Alcance del Caso de Prueba
TUC-01	UC1.2 Autenticar Usuario UC0 (Paso 1 Basic Path)	RF01 Positivo (Happy Path)	Autenticación válida de Voluntario y Administrador con redirección según rol detectado en `LoginView.xaml`.
TUC-02	UC1.2 Autenticar Usuario (1a, 2a) UC0 (Flujo Alterno 1A)	RF01 Negativo / Seguridad	Rechazo por credenciales inválidas/inexistentes y bloqueo temporal de cuenta al 3.er intento fallido consecutivo.
TUC-03	UC0 Gestionar Banco de Alimentos (Administración de Personal)	RF13, RNF04 Positivo / Negativo	Creación, edición y eliminación (CRUD) de cuentas de personal voluntario con ventana modal de confirmación en WPF.
TUC-04	UC1.1 Gestionar donantes UC0 (Paso 2 Basic Path)	RF07, RF09 Positivo (Happy Path)	Registro exitoso de nuevo donante en Oracle/Neo4j y vinculación de donación entrante al historial de aportaciones.

ID TUC	Caso de Uso Asociado (EA)	Req. / Escenario	Objetivo y Alcance del Caso de Prueba
TUC-05	UC1.1 Gestionar donantes (Escenarios Alternos 1a, 2a, 3a)	RF07, Nuevo RF14 Negativo / Excepción	Validación de datos inválidos (1a), detección de donante duplicado (2a) y omisión de campos obligatorios (3a).
TUC-06	UC1.3 Registrar ingresos de insumos UC0 (Paso 3 Basic Path)	RF02, RNF02, RNF12 Positivo (Happy Path)	Captura en WPF y persistencia ACID en Oracle 21c XE (vía Fachada/Dapper) y Neo4j 5 de un nuevo lote perecedero en ≤ 3 clics.
TUC-07	UC1.3 Registrar ingresos (1a, 2a) UC0 (Flujos Alternos 2A y 3A)	Nuevo RF14, RF02 Negativo / Bloqueo	Rechazo automático con Rollback en Oracle por fecha de caducidad vencida (2A) y bloqueo de botón en WPF por letras en cantidad (3A).
TUC-08	UC1.3 Registrar ingresos de insumos (Mantenimiento de Inventario)	RF03, Nuevo RF14 Positivo / Negativo	Modificación de datos de un lote en `Gestión de Lote` (WPF) por error de captura previa.
TUC-09	UC0 Gestionar Banco de Alimentos (Control de Mermas)	RF04, Nuevo RF15, RNF04 Positivo / Negativo	Baja de productos caducados o dañados exigiendo clasificar y registrar el motivo específico de merma.
TUC-10	UC0 Gestionar Banco de Alimentos (Consulta de Almacén)	RF05, RNF07 Positivo / Rendimiento	Búsqueda filtrada de insumos con Dapper por nombre, categoría y fecha de caducidad con respuesta ≤ 2 segundos.
TUC-11	UC0 (Paso 4 Basic Path) Consulta Semafórica y Simulación	RF06 Corr., RNF01, RNF03 Positivo / Usabilidad	Visualización del código de colores semafórico en WPF (Rojo, Amarillo, Verde), fuente ≥ 14pt y proyección en `Simulación`.
TUC-12	UC1.5 Procesar entrega logística (Padrón de Beneficiarios)	RF08, Nuevo RF14 Positivo / Negativo	Registro de beneficiarios con representante familiar y validación numérica del número de dependientes en WPF.
TUC-13	UC1.5 Procesar entrega logística UC0 (Paso 5 Basic Path)	RF10, RNF11, RNF13 Positivo / Integración	Salida de recursos guiada por regla FIFO, notificación Observer y mapeo de ruta Donante–Beneficiario en Neo4j 5.
TUC-14	UC1.5 Procesar entrega (1a, 2a) UC0 (Flujo Alterno 4A)	RF10 Negativo / Regla FIFO	Bloqueo por intento de violación FIFO (sacar lote nuevo existiendo antiguo) y autoselección del lote correcto.
TUC-15	UC1.5 Procesar entrega logística (Retornos y Cancelaciones)	Nuevo RF16, RF10 Positivo / Negativo	Restauración de insumos al inventario por entregas canceladas o devoluciones recuperando su posición FIFO original.

ID TUC	Caso de Uso Asociado (EA)	Req. / Escenario	Objetivo y Alcance del Caso de Prueba
TUC-16	UC1.4 Generar reportes de stock UC0 (Paso 6 Basic Path)	RF11, RF12, RNF08 Positivo / Rendimiento	Consulta de stock agrupado por categorías y exportación de reportes PDF (`QuestPDF`) y CSV en ≤ 5 segundos (10,000 registros).
TUC-17	UC1.4 Generar reportes de stock (Escenarios Alternos 1a, 2a, 3a)	RF11, RF12 Negativo / Excepción	Validación de filtros de fecha inválidos (1a), categoría inexistente (2a) y periodo sin movimientos registrados (3a).
TUC-18	UC0 (Paso 7 y Flujo Alterno 5A) Confiabilidad y Recursos	RNF05, RNF06, RNF09, RNF10 Confiabilidad / Sistema	Recuperación ACID tras apagón o cierre inesperado (5A), respaldo local cada 24h, RAM ≤ 512 MB en .NET 10 y cierre seguro de sesión.

1. BLOQUE DE AUTENTICACIÓN Y SEGURIDAD (UC1.2 & UC0)

TUC-01. Autenticación exitosa de Voluntario y Administrador (Basic Path)

ID Test Case:	**TUC-01**	**Caso de Uso Asociado:**	**UC1.2:** Autenticar Usuario (Basic Path) **UC0:** Gestionar Banco de Alimentos (Paso 1)
Requerimientos / UI:	RF01, RNF10 | UI WPF: `LoginView.xaml`, `Autenticacion Exitosa`, `DashboardView.xaml`	**Tipo / Prioridad:**	Escenario Positivo (Happy Path) | Prioridad: **Alta**
Precondiciones:	1. El ejecutable WPF del SGRC (.NET 10) está instalado y en ejecución local. 2. Los contenedores Docker de Oracle Database 21c XE y Neo4j 5 están activos. 3. Las cuentas `voluntario1` y `admin` existen activas en la tabla `Usuario`.		
Postcondiciones:	Sesión activa iniciada y acceso concedido a la interfaz principal WPF (`MainWindow - Voluntario` o `Administrador`) con los privilegios correspondientes al rol autenticado.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Iniciar la aplicación WPF SGRC para desplegar la ventana inicial `LoginView` y capturar las credenciales del Voluntario en las cajas de texto.	`txtUser = "voluntario1"` `txtPwd = "Vol2026!"`	Los controles de texto en WPF aceptan la entrada; el control `txtPwd` enmascara los caracteres de la contraseña.

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
2	Hacer clic en el botón `ENTRAR AL SISTEMA` y posteriormente presionar `OK` en la ventana emergente.	Clic en botón `ENTRAR AL SISTEMA` y clic en `OK`	El sistema valida los datos en Oracle Database 21c XE mediante Dapper, despliega el modal `Autenticacion Exitosa` (*"Bienvenido al sistema. Rol detectado: Voluntario"*) y abre `DashboardView`.
3	Hacer clic en `Cerrar Sesion` para volver a `LoginView`, ingresar las credenciales del Administrador y presionar `ENTRAR AL SISTEMA`.	`txtUser = "admin"` `txtPwd = "Admin2026!"`	El sistema muestra el modal `Autenticacion Exitosa` con *"Rol detectado: Administrador"* y carga el panel con permisos administrativos completos.

TUC-02. Autenticación fallida y bloqueo temporal al 3.er intento consecutivo (Flujos 1a, 2a y 1A)

ID Test Case:	**TUC-02**	**Caso de Uso Asociado:**	**UC1.2:** Autenticar Usuario (Alternos 1a y 2a) **UC0:** Escenario Alterno `1A. Autenticación_Fallida`
Requerimientos / UI:	RF01 | UI WPF: `LoginView.xaml`, `Error de Autenticacion`	**Tipo / Prioridad:**	Escenario Negativo / Seguridad | Prioridad: **Alta**
Precondiciones:	1. Aplicación WPF SGRC abierta en la pantalla `LoginView`. 2. Cuenta `voluntario1` activa con contador de intentos fallidos en `0`.		
Postcondiciones:	El acceso no autorizado es denegado en todos los intentos; tras el 3.er error consecutivo la cuenta queda bloqueada temporalmente en el sistema y no se inicia sesión.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Ingresar un formato de usuario inválido o dejar campos vacíos en `LoginView` y presionar `ENTRAR AL SISTEMA` (UC1.2 Escenario Alterno 1a).	`txtUser = ""` (vacío) `txtPwd = "123"`	El sistema marca en WPF que los datos son inválidos e impide el envío hasta que el usuario corrija el formato de captura (1a1, 1a2).
2	Ingresar un usuario existente con contraseña incorrecta y pulsar `ENTRAR AL SISTEMA` (Intento fallido 1 — UC1.2 Escenario Alterno 2a).	`txtUser = "voluntario1"` `txtPwd = "ClaveErronea01"`	El sistema valida en la BD y despliega el modal `Error de Autenticacion`: *"Usuario o contraseña incorrectos. Por favor, intente de nuevo."* Al pulsar `OK` regresa a `LoginView`.

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
3	Repetir el ingreso de contraseña errónea sobre `voluntario1` por segunda y tercera ocasión consecutiva (UC0 Flujo Alterno 1A).	Intento 2: `"ClaveErronea02"` Intento 3: `"ClaveErronea03"`	Al tercer error consecutivo, el sistema bloquea temporalmente la cuenta `voluntario1` y muestra alerta de cuenta bloqueada por seguridad.
4	Intentar iniciar sesión inmediatamente con la contraseña correcta de `voluntario1` mientras el bloqueo temporal está activo.	`txtUser = "voluntario1"` `txtPwd = "Vol2026!"`	El sistema mantiene el rechazo de acceso indicando que la cuenta se encuentra temporalmente bloqueada.

TUC-03. Gestión CRUD de cuentas para personal voluntario y ventana de confirmación

ID Test Case:	**TUC-03**	**Caso de Uso Asociado:**	**UC0:** Gestionar Banco de Alimentos (Gestión Administrativa de Cuentas)
Requerimientos / UI:	RF13, RNF04 | UI WPF: `DashboardView.xaml (Admin)`, Modal Confirmación	**Tipo / Prioridad:**	Escenario Positivo / Negativo | Prioridad: **Alta**
Precondiciones:	Usuario autenticado con rol de `Administrador` (`USR-ADM-01`) y cuenta temporal `USR-VOL-02 ("vol_temp")` registrada en Oracle Database 21c XE.		
Postcondiciones:	Nueva cuenta de voluntario creada y editada en tablas `Usuario` / `Voluntario`; cuenta temporal eliminada únicamente tras confirmar el diálogo emergente en WPF (RNF04).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	En el panel de administración de usuarios, capturar los datos para dar de alta una nueva cuenta de voluntario y guardar.	`nombreCompleto = "Pedro Ramírez"` `correo = "pramirez@sgrc.org"` `contraseña = "Pedro2026!"` `turnoAsignado = "Vespertino"`	El sistema crea el registro mediante Dapper en las tablas `Usuario` y `Voluntario` en Oracle y lo muestra en el listado activo.
2	Seleccionar el registro de `Pedro Ramírez`, modificar su turno asignado y horas acumuladas, y guardar los cambios.	`turnoAsignado = "Matutino"` `horasAcumuladas = 8`	El sistema actualiza los atributos de la subclase `Voluntario` en Oracle y refresca el `DataGrid` en pantalla.
3	Seleccionar la cuenta `vol_temp`, presionar el botón `Eliminar` y en la ventana emergente de confirmación presionar `Cancelar`.	Acción: `Eliminar` → Modal RNF04: `Cancelar`	El sistema despliega obligatoriamente la ventana emergente de confirmación (RNF04); al cancelar, la cuenta no se elimina.

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
4	Presionar nuevamente `Eliminar` sobre `vol_temp` y confirmar la acción pulsando `Aceptar / OK` en la ventana emergente.	Acción: `Eliminar` → Modal RNF04: `OK`	El registro se elimina de la base de datos y sus credenciales quedan revocadas de inmediato.

2. BLOQUE DE GESTIÓN DE DONANTES (UC1.1 & UC0)

TUC-04. Registro de nuevo donante y vinculación de donaciones al historial (Basic Path)

ID Test Case:	**TUC-04**	**Caso de Uso Asociado:**	**UC1.1:** Gestionar donantes (Basic Path) **UC0:** Gestionar Banco de Alimentos (Paso 2)
Requerimientos / UI:	RF07, RF09, RNF12, RNF13 | UI WPF: `DashboardView.xaml`, `LogisticaView.xaml`	**Tipo / Prioridad:**	Escenario Positivo (Happy Path) | Prioridad: **Alta**
Precondiciones:	Usuario autenticado con rol de `Administrador`. Conexión activa con los contenedores Docker de Oracle Database 21c XE y Neo4j 5.		
Postcondiciones:	Datos del donante almacenados de forma segura en la tabla `Donante` (Oracle) y como nodo en Neo4j 5, con sus lotes donados vinculados a su historial de aportaciones (Postcondición 2 de UC0).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	El Administrador abre el módulo `Gestionar donantes` en WPF e ingresa los datos del nuevo donante (UC1.1 Paso 1).	`nombreOrganización = "Fundación Alimentar México A.C."` `telefono = "5551234567"` `correo = "contacto@alimentarmx.org"`	El sistema valida el formato de los datos y verifica en la base de datos que no sea un registro duplicado (UC1.1 Paso 2).
2	El Administrador confirma la acción presionando `Guardar` (UC1.1 Paso 3).	Clic en botón `Guardar`	El donante queda persistido con `idDonante = 101` en Oracle vía Dapper y se crea su nodo origen en Neo4j 5 vía `Neo4j.Driver`.
3	Registrar una donación entrante (`LOT-501` de Leche 1L) seleccionando a `Fundación Alimentar México A.C.` como donante origen y consultar su perfil (RF09).	`idDonante = 101` `idLote = 501` (Cantidad: `50 u.`)	El lote queda vinculado mediante llave foránea al perfil del donante en Oracle, crea la relación en Neo4j y aparece listado en su historial de aportaciones.

TUC-05. Validación de excepciones en Donantes: Datos inválidos, duplicados y campos vacíos (1a, 2a, 3a)

ID Test Case:	**TUC-05**	**Caso de Uso Asociado:**	**UC1.1:** Gestionar donantes (Escenarios Alternos `1a`, `2a` y `3a`)
Requerimientos / UI:	RF07, Nuevo RF14 | UI WPF: Formulario de Gestión de Donantes	**Tipo / Prioridad:**	Escenario Negativo / Validación | Prioridad: **Media**
Precondiciones:	Usuario autenticado con rol de `Administrador`; el donante `"Fundación Alimentar México A.C."` ya existe registrado en la base de datos.		
Postcondiciones:	El sistema impide el almacenamiento de registros con teléfono alfabético, donantes duplicados o formularios incompletos, preservando la integridad del catálogo.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Ingresar datos inválidos en el formulario de donante (letras en el campo telefónico y correo sin formato válido — Escenario Alterno 1a).	`nombreOrganización = "Donante Test"` `telefono = "TelInvalidoABC"` `correo = "correosin_arroba"`	El sistema indica al Administrador en WPF que los datos son inválidos, resalta el campo erróneo y bloquea el guardado hasta su corrección (1a1, 1a2, 1a3).
2	Ingresar exactamente los mismos datos de un donante ya registrado y presionar `Guardar` (Escenario Alterno 2a).	`nombreOrganización = "Fundación Alimentar México A.C."` `correo = "contacto@alimentarmx.org"`	El sistema detecta que el registro es duplicado y notifica al Administrador que ya existe ese donante en la base de datos (2a1, 2a2).
3	Intentar confirmar el guardado dejando vacío el campo de nombre de organización o teléfono (Escenario Alterno 3a).	`nombreOrganización = ""` (vacío) `telefono = "5559876543"`	El sistema marca error por dato obligatorio faltante y solicita al Administrador rellenar el campo vacío antes de continuar (3a1, 3a2).

3. BLOQUE DE INVENTARIO PERECEDERO, VALIDACIONES EN WPF Y MERMAS (UC1.3 & UC0)

TUC-06. Registro exitoso de ingreso de insumo perecedero en ≤ 3 clics (Basic Path & Escenario del Robot)

ID Test Case:	**TUC-06**	**Caso de Uso Asociado:**	**UC1.3:** Registrar ingresos de insumos (Basic Path) **UC0:** Gestionar Banco de Alimentos (Paso 3)
Requerimientos / UI:	RF02, RNF02, RNF12, RNF13 | UI WPF: `DashboardView.xaml`, `Registro Exitoso`, `InventarioView.xaml`	**Tipo / Prioridad:**	Escenario Positivo (Happy Path / Robot) | Prioridad: **Alta**
Precondiciones:	Usuario autenticado (`Voluntario` o `Administrador`) ubicado en el menú principal (`DashboardView`); el insumo pertenece a una categoría válida y existe el donante `DON-101`.		
Postcondiciones:	Incremento en el stock global, creación del registro en las tablas `Insumo_Perecedero` y `Lote` en Oracle Database 21c XE garantizando propiedades ACID, y creación de la relación con el donante en Neo4j 5 (Postcondición 3 de UC0).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Desde el menú principal (`DashboardView`), navegar hacia el formulario de registro de inventario contando el número de clics (RNF02).	Clic 1: Botón `Registrar Ingreso` (o vía `Inventario` → `Nuevo`)	El formulario WPF de captura se despliega en un máximo de 1 a 2 clics (cumpliendo la regla de ≤ 3 clics de RNF02).

2	Capturar los datos completos del insumo perecedero y confirmar el ingreso mediante la Fachada (UC1.3 Pasos 1 y 2).	`nmbreInsumo = "Leche Entera Pasteurizada 1L"`
		`categoria = "Lácteos"`
		`idLote = "LOT-503"`
		`cantidad = 120`
		`fecha_caducidad = Fecha_Actual + 25 días`
		`Donante = DON-101`
			El sistema valida los datos, ejecuta el `COMMIT` ACID en Oracle Database con Dapper, registra el vínculo en Neo4j 5 y muestra la ventana modal `Registro Exitoso`; al pulsar `OK` actualiza el stock en el `DataGrid` de inventario.

TUC-07. Bloqueo por fecha de caducidad vencida (Rollback 2A), letras en cantidad en WPF (3A) y datos faltantes

ID Test Case:	**TUC-07**	**Caso de Uso Asociado:**	**UC1.3:** Registrar ingresos de insumos (Alternos `1a` y `2a`) **UC0:** Escenarios Alternos `2A. Caducidad_Invalida` y `3A. Error_Captura`
Requerimientos / UI:	Nuevo RF14, RF02, RNF12 | UI WPF: Formulario de Registro de Inventario (XAML)	**Tipo / Prioridad:**	Escenario Negativo / Reglas de Validación | Prioridad: **Crítica**
Precondiciones:	Usuario autenticado con el formulario WPF de registro de ingreso de inventario abierto en pantalla.		
Postcondiciones:	Ningún lote caducado ni cantidad no numérica se inserta en Oracle Database 21c XE; la transacción inválida ejecuta Rollback y el inventario permanece íntegro.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Capturar un insumo ingresando una fecha de caducidad anterior a la fecha actual e intentar guardar (UC1.3 Alterno 1a / UC0 Alterno 2A).	`nmbreInsumo = "Yogurt Natural"` `categoria = "Lácteos"` `cantidad = 40` `fecha_caducidad = "01/01/2024"` (Vencida)	El sistema detecta que el producto ya está caducado, aborta el registro ejecutando `Rollback` en Oracle Database y lanza la alerta de *"Rechazo automático: No es posible donar ni registrar productos caducados"* (2A).
2	Escribir caracteres alfabéticos o símbolos en el campo numérico de `cantidad` (UC0 Alterno 3A y Nuevo RF14).	`cantidad = "Cincuenta50ABC"`	La validación en WPF (`MaterialDesignThemes`) resalta inmediatamente la casilla en **color rojo** y **desactiva el botón de guardado**, impidiendo el envío (3A).
3	Corregir la cantidad a `40` y la fecha a un valor futuro válido, pero dejar vacío el campo de nombre o categoría e intentar confirmar (UC1.3 Alterno 2a).	`nmbreInsumo = ""` (vacío) `cantidad = 40` `fecha_caducidad = Fecha_Actual + 15 días`	El sistema marca que falta un dato obligatorio y solicita al usuario rellenarlo antes de procesar la transacción (2a1, 2a2).

TUC-08. Modificación de datos de un lote en inventario por error de captura

ID Test Case:	**TUC-08**	**Caso de Uso Asociado:**	**UC1.3:** Registrar ingresos de insumos (Corrección de Captura) **UC0:** Gestionar Banco de Alimentos
Requerimientos / UI:	RF03, Nuevo RF14 | UI WPF: `InventarioView.xaml`, `Gestión de Lote`	**Tipo / Prioridad:**	Escenario Positivo / Negativo | Prioridad: **Media**

Precondiciones:	El Voluntario está autenticado en `InventarioView` y el lote `LOT-503` (`cantidad = 120`) se encuentra registrado en el almacén.
Postcondiciones:	Los atributos del lote corregido quedan actualizados en Oracle Database 21c XE y reflejados en el `DataGrid` de inventario en WPF.

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Seleccionar el lote `LOT-503` en `InventarioView` y presionar el botón `Editar / Eliminar` para abrir el estado `Gestión de Lote`.	Selección: `LOT-503` → Clic en `Editar / Eliminar`	Se despliega la ventana `Gestión de Lote` en WPF con los valores actuales del registro cargados en los controles de edición.
2	Modificar la cantidad de `120` a `115` (por ajuste de error de captura) y presionar `Confirmar Acción`.	`cantidad = 115` → Clic en `Confirmar Acción`	El sistema valida que el nuevo valor cumpla con el Nuevo RF14, ejecuta el `UPDATE` en Oracle Database vía Dapper y retorna a `InventarioView` mostrando `115` unidades.

TUC-09. Baja de productos caducados y registro clasificado del motivo de merma

ID Test Case:	**TUC-09**	**Caso de Uso Asociado:**	**UC0:** Gestionar Banco de Alimentos (Control de Bajas y Mermas)
Requerimientos / UI:	RF04, Nuevo RF15, RNF04 | UI WPF: `InventarioView.xaml`, `Gestión de Lote`	**Tipo / Prioridad:**	Escenario Positivo / Negativo | Prioridad: **Alta**
Precondiciones:	Usuario autenticado con rol de `Administrador`; existe el lote `LOT-504` identificado como caducado o dañado en almacén.		
Postcondiciones:	El producto es dado de baja del inventario disponible en la tabla `Lote` y queda registrado en la tabla `Merma` con su cantidad, fecha y motivo específico para trazabilidad de pérdidas.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Seleccionar `LOT-504` en `InventarioView`, abrir `Gestión de Lote` e intentar ejecutar su baja sin especificar el motivo de la merma.	`Motivo = ""` (Sin seleccionar)	El sistema rechaza la operación exigiendo clasificar y registrar obligatoriamente el motivo específico de la merma (Nuevo RF15).
2	Clasificar el motivo de la pérdida, confirmar la acción y aceptar la ventana emergente de confirmación (RNF04).	`idLote = "LOT-504"` `Cantidad = 25` `Motivo = "Caducidad natural en anaquel"` Modal RNF04: `Confirmar / OK`	El sistema muestra primero el modal de confirmación en WPF (RNF04); al aceptar, da de baja el lote del stock activo e inserta el registro histórico en la tabla `Merma`.

TUC-10. Búsqueda filtrada de productos por nombre, categoría y caducidad (≤ 2 segundos)

ID Test Case:	**TUC-10**	**Caso de Uso Asociado:**	**UC0:** Gestionar Banco de Alimentos (Búsqueda en Almacén)
Requerimientos / UI:	RF05, RNF07 | UI WPF: `InventarioView.xaml`	**Tipo / Prioridad:**	Escenario Positivo / Rendimiento | Prioridad: **Alta**
Precondiciones:	Usuario autenticado en `InventarioView` con lotes cargados en múltiples categorías (`Lácteos`, `Frutas y Verduras`, `Granos y Cereales`).		
Postcondiciones:	El `DataGrid` de inventario en WPF presenta los registros coincidentes en un tiempo de respuesta ≤ 2.0 segundos sin modificar los datos almacenados.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Ingresar en el filtro de búsqueda de `InventarioView` el nombre del producto y medir el tiempo de respuesta con Dapper.	Filtro Nombre: `"Leche"`	El sistema procesa y muestra en pantalla los lotes `LOT-501`, `LOT-502` y `LOT-503` en un tiempo **≤ 2.0 segundos** (RNF07).
2	Filtrar sucesivamente por categoría de alimento y por fecha de caducidad próxima.	Filtro Categoría: `"Lácteos"` Filtro Caducidad: `≤ Fecha_Actual + 7 días`	La tabla acota los resultados en ≤ 2.0 segundos mostrando únicamente `LOT-501` (+2 días) y `LOT-502` (+6 días).

TUC-11. Consulta del semáforo de urgencia de caducidad y proyección en vista Simulación

ID Test Case:	**TUC-11**	**Caso de Uso Asociado:**	**UC0:** Gestionar Banco de Alimentos (Paso 4 Basic Path)
Requerimientos / UI:	RF06 Corregido, RNF01, RNF03 | UI WPF: `DashboardView.xaml`, `InventarioView.xaml`, `Simulación`	**Tipo / Prioridad:**	Escenario Positivo / Usabilidad | Prioridad: **Alta**
Precondiciones:	Existencia de lotes con diferentes horizontes de vencimiento en Oracle: `LOT-501` (+2 días), `LOT-502` (+6 días) y `LOT-503` (+25 días).		
Postcondiciones:	La interfaz gráfica WPF se actualiza mostrando las alertas de caducidad semafóricas y tipografía ≥ 14pt sin alterar los datos subyacentes en la base de datos (Postcondición 4 de UC0).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	El Voluntario abre `InventarioView` y consulta la columna de estado de urgencia y el tamaño de fuente de la tabla (UC0 Paso 4).	Consulta visual de `LOT-501`, `LOT-502` y `LOT-503`	Se muestra el código de colores semafórico en WPF (RNF01) con fuente ≥ 14pt (RNF03): `LOT-501` en **Rojo (Crítico)**, `LOT-502` en **Amarillo (Preventivo)** y `LOT-503` en **Verde (Óptimo)**.

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
2	En `DashboardView`, hacer clic en `Simular Fecha` para abrir la ventana `Simulación`, adelantar la fecha proyectada `+5 días` y luego pulsar `OK`.	`Fecha_Simulada = Fecha_Actual + 5 días` → Clic en `OK`	Durante la simulación, `LOT-502` cambia a color **Rojo (Crítico)** al restarle solo 1 día de vigencia; al pulsar `OK` se retorna al estado normal en `DashboardView`.

4. BLOQUE DE ENTREGAS LOGÍSTICAS, MOTOR FIFO Y GRAFOS NEO4J 5 (UC1.5 & UC0)

TUC-12. Registro de beneficiarios y validación numérica de dependientes familiares en WPF

ID Test Case:	**TUC-12**	**Caso de Uso Asociado:**	**UC1.5:** Procesar entrega logística (Padrón de Beneficiarios) **UC0:** Gestionar Banco de Alimentos
Requerimientos / UI:	RF08, Nuevo RF14, RNF13 | UI WPF: `LogisticaView.xaml`	**Tipo / Prioridad:**	Escenario Positivo / Negativo | Prioridad: **Media**
Precondiciones:	Voluntario autenticado en el módulo `LogisticaView` con conexión activa a Oracle Database 21c XE y Neo4j 5.		
Postcondiciones:	Perfil del beneficiario creado en la tabla `Beneficiario` en Oracle y como nodo receptor en Neo4j 5 listo para recibir entregas logísticas.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Intentar capturar en el formulario WPF de beneficiarios caracteres alfabéticos dentro del campo `numDependientes`.	`nombreRepresentante = "María Guadalupe Hernández"` `numDependientes = "CincoPersonas"`	El validador de WPF (Nuevo RF14) resalta la casilla en rojo y desactiva el botón de guardado.
2	Corregir el campo colocando el número entero `5` y confirmar el registro del beneficiario (RF08).	`nombreRepresentante = "María Guadalupe Hernández"` `numDependientes = 5`	El beneficiario se guarda con `idBeneficiario = 201` en Oracle Database vía Dapper y se genera el nodo `(:Beneficiario {id: 201})` en Neo4j 5.

TUC-13. Salida de recursos guiada por FIFO, notificación Observer y mapeo en Neo4j 5 (Basic Path)

ID Test Case:	**TUC-13**	**Caso de Uso Asociado:**	**UC1.5:** Procesar entrega logística (Basic Path) **UC0:** Gestionar Banco de Alimentos (Paso 5)

Requerimientos / UI:	RF10, RNF11, RNF12, RNF13 | UI WPF: `LogisticaView.xaml`, `NOTIFICACION`	**Tipo / Prioridad:**	Escenario Positivo / Integración Políglota | Prioridad: **Alta**
Precondiciones:	Usuario autenticado; existencia de stock suficiente en inventario para `INS-01` con tres lotes activos: `LOT-501` (+2 días, 50 u.), `LOT-502` (+6 días, 80 u.) y `LOT-503` (+25 días, 115 u.); beneficiario `BEN-201` registrado.		
Postcondiciones:	El inventario se descuenta correctamente de Oracle respetando el método FIFO, el método Observer notifica la actualización y la ruta de entrega entre el donante y el beneficiario queda mapeada en el motor de grafos Neo4j 5 (Postcondición 5 de UC0 y Postcondición de UC1.5).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado

1	En `LogisticaView`, seleccionar el insumo `Leche Entera 1L` y al beneficiario `BEN-201` para registrar una entrega de `30 unidades`.	`idInsumo = 1`
		`idBeneficiario = 201`
		`cantidadEntregada = 30`
			El motor FIFO selecciona automáticamente el lote más próximo a caducar: `LOT-501` (vence en +2 días, aportado por `DON-101`).

2	Confirmar el procesamiento de la entrega logística a través de la Fachada (UC1.5 Pasos 1 y 2 / UC0 Paso 5).	Clic en botón `Registrar Entrega`	1. Se descuenta `LOT-501` en Oracle (quedando en `20 u.`) y se crea el registro en `Entrega_Logistica`. 2. El método `Observer` dispara la ventana/aviso `NOTIFICACION` indicando que se modificaron los datos. 3. En Neo4j 5 se traza la relación exacta `(DON-101)-[:APORTO]->(LOT-501)-[:ENTREGADO_A]->(BEN-201)`.

TUC-14. Bloqueo por intento de violación de la regla FIFO y falta de stock (Flujos 4A, 1a y 2a)

ID Test Case:	**TUC-14**	**Caso de Uso Asociado:**	**UC1.5:** Procesar entrega logística (Alternos `1a` y `2a`) **UC0:** Escenario Alterno `4A. Violación_Fifo`
Requerimientos / UI:	RF10, RF06 Corregido | UI WPF: `LogisticaView.xaml`	**Tipo / Prioridad:**	Escenario Negativo / Regla de Negocio FIFO | Prioridad: **Crítica**
Precondiciones:	Voluntario autenticado en `LogisticaView`; el lote antiguo `LOT-501` (+2 días) aún cuenta con `20 unidades` disponibles y el lote nuevo `LOT-503` (+25 días) tiene `115 unidades`.		
Postcondiciones:	El sistema impide despachar el lote nuevo existiendo stock en el antiguo, autoselecciona el lote más próximo a vencer y evita guardar transacciones vacías.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Intentar seleccionar manualmente el lote reciente `LOT-503` (+25 días) para entregar `15 unidades` cuando `LOT-501` (+2 días) aún tiene `20 unidades` (UC0 Flujo Alterno 4A).	Selección manual: `LOT-503` `cantidadEntregada = 15`	El sistema **cancela la selección manual**, lanza la alerta *"Violación FIFO: Debe distribuirse primero el lote más próximo a caducar"* y **autoselecciona el lote** `LOT-501` (4A).
2	Intentar procesar una entrega sobre una categoría o insumo donde nadie ha donado nada / stock en cero (UC1.5 Escenario Alterno 1a).	`idInsumo = 99` (Sin donaciones / Stock = 0)	El sistema detecta que no existen lotes disponibles, no guarda ningún movimiento y notifica la falta de existencias (1a1, 1a2).
3	Solicitar la descarga/consulta de los datos de trazabilidad logística tras la notificación del método Observer (UC1.5 Escenario Alterno 2a).	Clic en consultar/descargar datos de red logística	El sistema entrega al usuario los datos actualizados de entradas, salidas y nodos asociados (2a1, 2a2).

TUC-15. Restauración de insumos al inventario por entregas canceladas o devoluciones

ID Test Case:	**TUC-15**	**Caso de Uso Asociado:**	**UC1.5:** Procesar entrega logística (Devoluciones) **UC0:** Gestionar Banco de Alimentos
Requerimientos / UI:	Nuevo RF16, RF10 | UI WPF: `LogisticaView.xaml`, `InventarioView.xaml`	**Tipo / Prioridad:**	Escenario Positivo / Integridad FIFO | Prioridad: **Alta**
Precondiciones:	Existe la entrega registrada en TUC-13 de `30 unidades` del lote `LOT-501` (+2 días), cuyo saldo restante en almacén es de `20 unidades`.		
Postcondiciones:	Las unidades devueltas se reintegran a `LOT-501` en Oracle Database 21c XE y el lote conserva su posición número 1 en la cola FIFO según su fecha de caducidad original.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	El Voluntario selecciona la entrega de `30 unidades` de `LOT-501` y registra su restauración al inventario por entrega cancelada/devolución (Nuevo RF16).	`idEntrega = 1` (Lote: `LOT-501`) `cantidadRestaurada = 30` Motivo: `"Entrega cancelada en ruta"`	El stock de `LOT-501` en Oracle se incrementa de `20` a `50 unidades` y se actualiza la relación en Neo4j 5.
2	Verificar en `InventarioView` y en el selector de salidas el orden de prioridad de `LOT-501` frente a `LOT-502` (+6 días) y `LOT-503` (+25 días).	Consulta de cola FIFO para `INS-01`	`LOT-501` mantiene su fecha de caducidad original (+2 días) y permanece en la **primera posición de salida obligatoria** del motor FIFO.

5. BLOQUE DE REPORTES DE STOCK (QUESTPDF / CSV) Y CONFIABILIDAD DEL SISTEMA (UC1.4 & UC0)

TUC-16. Consulta de stock por categoría y exportación de reportes PDF (QuestPDF) y CSV en ≤ 5 s (Basic Path)

ID Test Case:	**TUC-16**	**Caso de Uso Asociado:**	**UC1.4:** Generar reportes de stock (Basic Path) **UC0:** Gestionar Banco de Alimentos (Paso 6)
Requerimientos / UI:	RF11, RF12, RNF08 | UI WPF: `InventarioView.xaml`, Modal `Exito CSV`	**Tipo / Prioridad:**	Escenario Positivo / Rendimiento | Prioridad: **Alta**
Precondiciones:	Usuario autenticado como `Administrador`; existen registros de movimientos de inventario, logística y mermas en el periodo solicitado (incluyendo prueba de volumen con hasta 10,000 registros).		
Postcondiciones:	Visualización de métricas agrupadas por categoría en pantalla y archivo físico generado con `QuestPDF` (PDF) y `System.IO` (CSV) en el disco duro local en ≤ 5.0 segundos (Postcondición 6 de UC0 y Postcondición de UC1.4).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	El Administrador define los filtros de consulta (rango de fechas válido y agrupación por categoría de alimentos — UC1.4 Paso 1 y RF11).	Filtro: `Todas las categorías` Rango: `Mes actual`	El sistema procesa los datos en Oracle Database vía Dapper / GraphQL y muestra en pantalla el total de stock disponible agrupado por categoría y el resumen de mermas (UC1.4 Paso 2).
2	Hacer clic en el botón `Exportar CSV` y posteriormente exportar en formato `PDF` con `QuestPDF` sobre un volumen de hasta 10,000 registros (UC1.4 Paso 3, RF12 y RNF08).	Formato 1: `CSV` Formato 2: `PDF (QuestPDF)` Volumen: `10,000 registros`	Ambos reportes se generan y guardan en el disco local en un tiempo **≤ 5.0 segundos**; la interfaz WPF muestra el modal `Exito` (*"Reporte exportado dinamicamente! Guardado en: PATH"*).

TUC-17. Validación de excepciones al generar reportes de stock (Escenarios Alternos 1a, 2a y 3a)

ID Test Case:	**TUC-17**	**Caso de Uso Asociado:**	**UC1.4:** Generar reportes de stock (Escenarios Alternos `1a`, `2a` y `3a`)

Requerimientos / UI:	RF11, RF12 | UI WPF: Módulo de Reportes en `InventarioView.xaml` / `DashboardView.xaml`	**Tipo / Prioridad:**	Escenario Negativo / Excepción | Prioridad: **Media**
Precondiciones:	Usuario autenticado con rol de `Administrador` en la pantalla de generación de reportes de stock.		
Postcondiciones:	El sistema previene la generación de archivos vacíos o corruptos ante filtros incoherentes e informa claramente al Administrador para que ajuste los parámetros.		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Definir un filtro con rango de fechas inválido (fecha final anterior a la fecha inicial — UC1.4 Escenario Alterno 1a).	`Fecha_Inicio = "15/11/2026"` `Fecha_Fin = "01/11/2026"`	El sistema marca en WPF que hay un dato inválido en el filtro y solicita al Administrador corregir el rango cronológico (1a1, 1a2).
2	Solicitar el procesamiento de datos ingresando un parámetro de categoría o lote que no es posible o no existe en el catálogo (UC1.4 Escenario Alterno 2a).	`Categoria = "CategoríaInexistente99"`	El sistema indica que el parámetro solicitado no existe en la base de datos y permite al Administrador corregir el criterio (2a1, 2a2).
3	Solicitar la exportación de un rango histórico válido en el que no hubo ningún movimiento registrado (ej. año 2019 — UC1.4 Escenario Alterno 3a).	`Fecha_Inicio = "01/01/2019"` `Fecha_Fin = "31/01/2019"`	El sistema notifica al Administrador que no existe ningún registro con dichos datos para exportar y le permite buscar otro periodo (3a1, 3a2).

TUC-18. Persistencia ACID ante cierre inesperado (5A), respaldo 24h, límite de RAM en .NET 10 (≤ 512 MB) y cierre seguro

ID Test Case:	**TUC-18**	**Caso de Uso Asociado:**	**UC0:** Gestionar Banco de Alimentos (Paso 7 Basic Path y Escenario Alterno `5A. Cierre_Inesperado`)
Requerimientos / UI:	RNF05, RNF06, RNF09, RNF12 | UI WPF: `DashboardView.xaml`, Monitor .NET 10 CLR y Contenedor Oracle 21c XE	**Tipo / Prioridad:**	Escenario de Sistema / Confiabilidad y Recursos | Prioridad: **Alta**
Precondiciones:	Aplicación WPF SGRC operando sobre .NET 10 con sesión activa y transacciones previas confirmadas en Oracle Database 21c XE y Neo4j 5.		
Postcondiciones:	1. Ante cierre abrupto, las transacciones incompletas se revierten y los datos confirmados persisten íntegros (ACID). 2. Respaldo local generado cada 24h. 3. Al cerrar sesión normalmente, se cierran las conexiones a BD y se libera la memoria RAM en .NET 10 (Postcondiciones 1 y 7 de UC0).		

Paso	Pasos de Ejecución (Actividad)	Datos de Entrada	Resultado Esperado
1	Monitorear el consumo de memoria RAM del proceso `SGRC.exe` (.NET 10 / WPF) durante la ejecución de los casos TUC-01 a TUC-17 y verificar la generación del respaldo diario (RNF05, RNF09).	Ejecución continua en .NET 10 CLR + Tarea de Backup 24h	El consumo de memoria se mantiene fluido y **≤ 512 MB de RAM**; el archivo de respaldo local de la base de datos se genera automáticamente cada 24 horas.
2	Iniciar la captura de un movimiento sin confirmarlo y forzar el cierre inesperado del proceso de la aplicación simulando un apagón eléctrico (UC0 Flujo Alterno 5A y RNF06).	Interrupción forzada del proceso (`End Task / taskkill /F`) con transacción abierta	Oracle Database 21c XE cancela la transacción incompleta; al reiniciar el SGRC, el sistema recupera el último estado válido garantizando 100% de integridad ACID sin pérdida de las capturas previas del día (5A).
3	Con la aplicación reiniciada y operativa, hacer clic en el botón `Cerrar Sesion` en `DashboardView` (UC0 Paso 7 Basic Path).	Clic en botón `Cerrar Sesion`	La sesión se cierra de manera segura regresando a `LoginView`, cerrando las conexiones activas a Oracle y Neo4j y liberando la memoria RAM consumida (Postcondiciones 1 y 7).